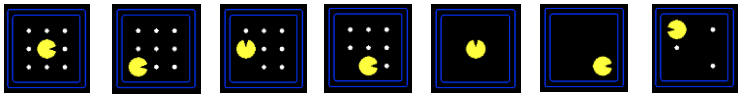
- UNIT – II: Searching: Searching for solutions, uniformed search strategies – Breadth first search, depth limited Search. Search with partial information (Heuristic search) Greedy best first search, A* search, Memory - bounded heuristic search, Local search algorithms- Hill climbing, Simulated annealing search, Local beam search, Genetic algorithms

Search Problems

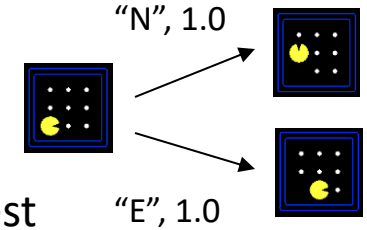


Search Problems

- A search problem consists of:
  - A state space



  - A successor function (with actions, costs)



  - A start state and a goal test

- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

Goal directed Agent

- A goal directed agent needs to achieve certain goals.
- Many problems can be represented as a set of states and a set of rules of how one state is transformed to another
- The agent must choose a sequence of actions to achieve the desired goal.

Each state is an abstract representation of the agent's environment. It is an abstraction that denotes a configuration of the agent.

- Initial state : The description of the starting configuration of the agent
- An action/ operator takes the agent from one state to another state. A state can have a number of successor states.
- A *plan* is a sequence of actions.

- *Problem formulation* means choosing a relevant set of states to consider, and a feasible set of operators for moving from one state to another.
- *Search* is the process of *imagining* sequences of operators applied to the initial state, and checking which sequence reaches a goal state.

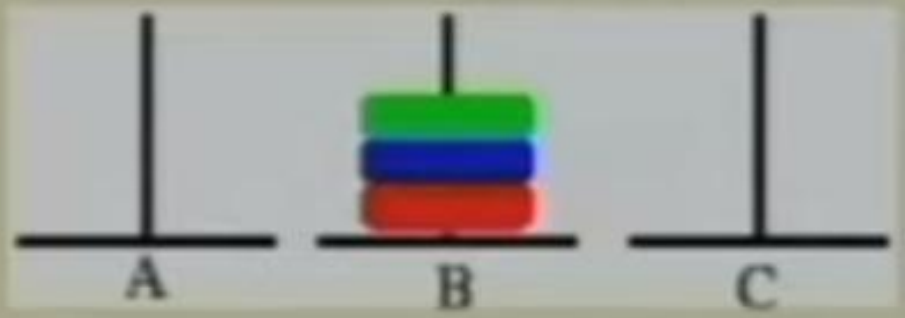
Pegs and Disks

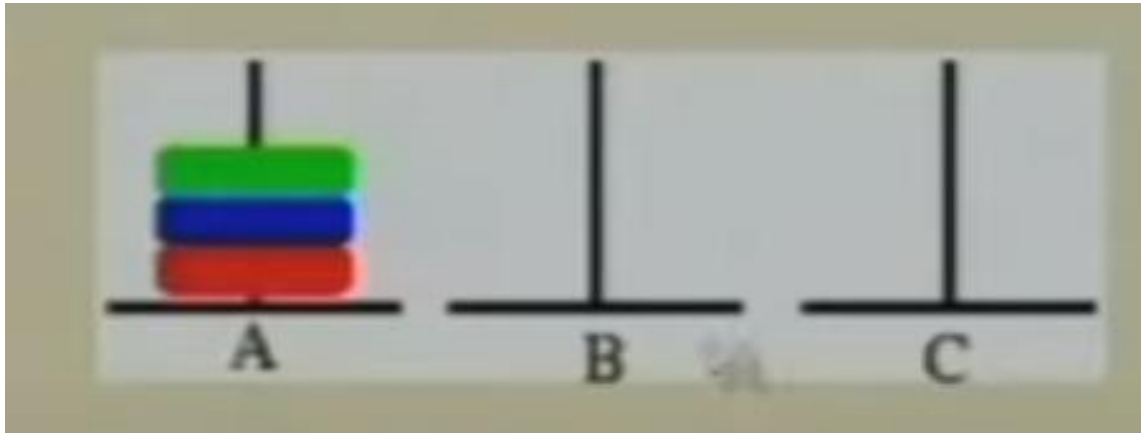
- Consider the following problem. We have 3 pegs and 3 disks.
- Operators: one may move the topmost disk on any needle to the topmost position to any other needle

Initial state



Goal configuration





A -> C

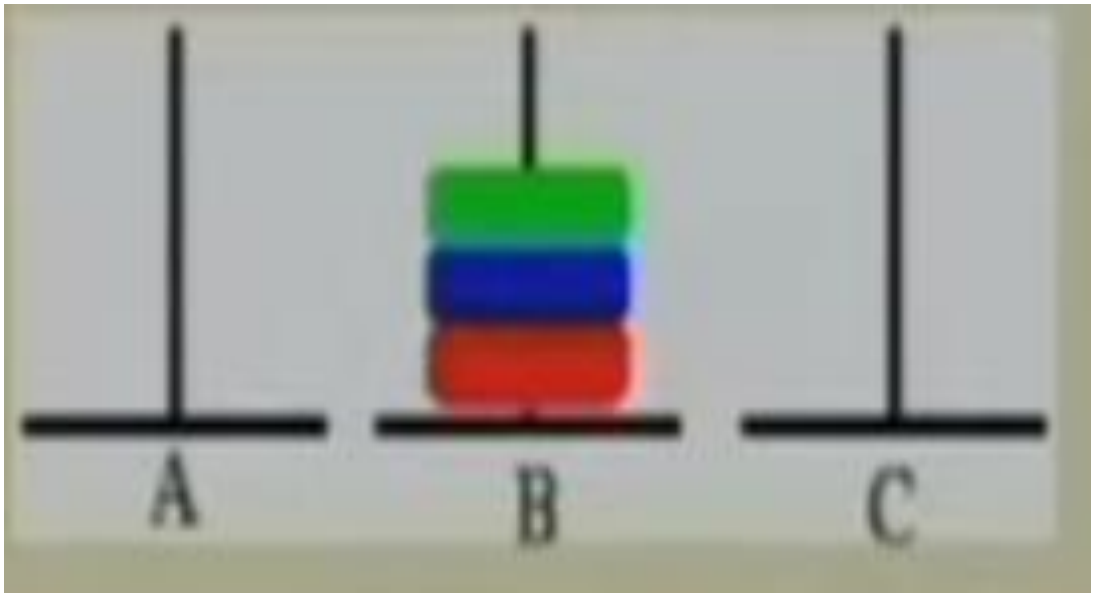
A->B

B->C

A->B

C->B

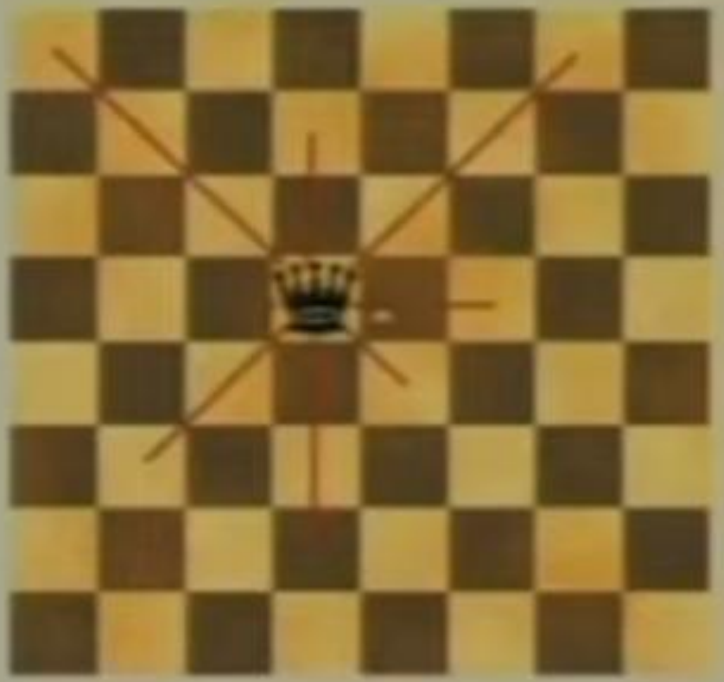
C->B



8 queens

- Place 8 queens on a chessboard so that no two queens are in the same row, column or diagonal





8 queens solution



N queens problem formulation 1

- States : Any arrangement of 0 to 8 queens on the board
- Initial state : 0 queens on the board
- Successor function : Add a queen in any square
- Goal test : 8 queens on the board, none are attacked



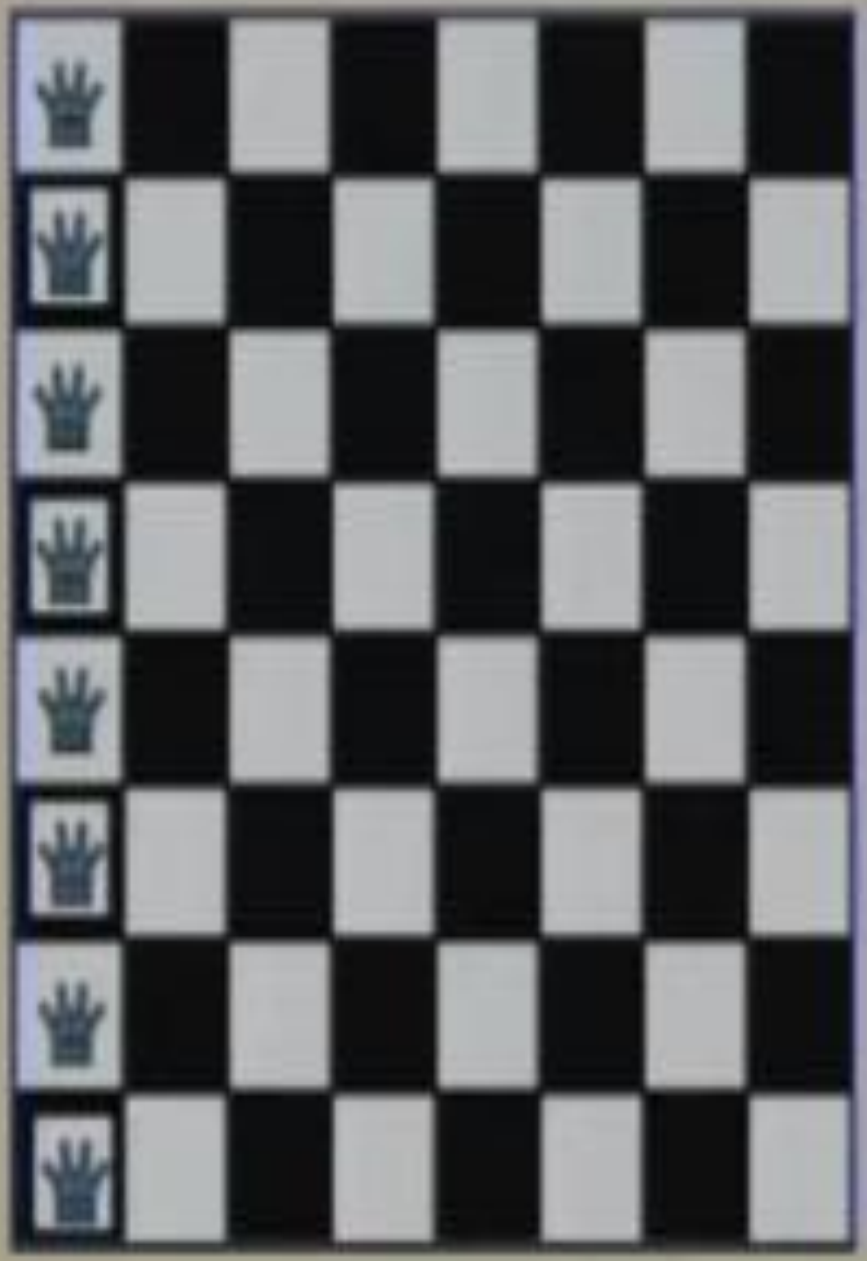


64 successors



N queens problem formulation 2

- States : Any arrangement of 8 queens on the board
- Initial state : All queens are at column 1
- Successor function : Change the position of any one queen
- Goal test : 8 queens on the board, none are attacked





N queens problem formulation 3

- States : Any arrangement of k queens in the first k rows such that none are attacked
- Initial state : 0 queens on the board
- Successor function : Add a queen to the (k+1)th row so that none are attacked.
- Goal test : 8 queens on the board, none are attacked





Problem Definition - Example, 8 puzzle



Initial State



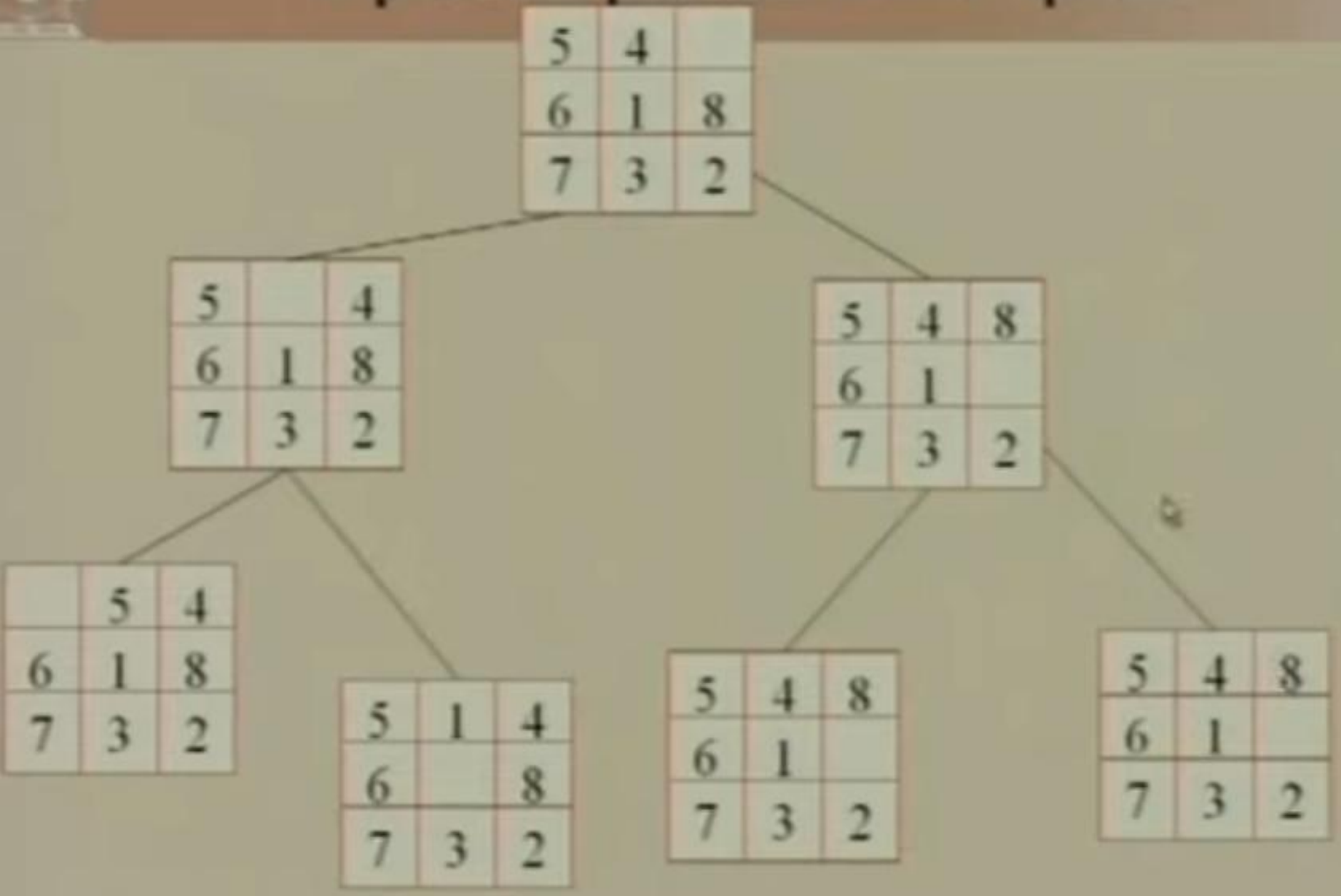
Goal State

Problem Definition - Example, 8 puzzle



- States
  - A description of each of the eight tiles in each location that it can occupy.
- Operators/Action
  - The blank moves left, right, up or down
- Goal Test
  - The current state matches a certain state (e.g. one of the ones shown on previous slide)
- Path Cost
  - Each move of the blank costs 1

8-puzzle partial state space



Search strategies

- A search strategy is defined by picking the order of node expansion
- Strategies are evaluated along the following dimensions:
  - completeness: does it always find a solution if one exists?
  - time complexity: number of nodes generated
  - space complexity: maximum number of nodes in memory
  - optimality: does it always find a least-cost solution?
  - systematicity: does it visit each state at most once?

- Time and space complexity are measured in terms of
  - *b:* maximum branching factor of the search tree
  - *d:* depth of the least-cost solution
  - *m*: maximum depth of the state space (may be ∞)

Types of search algorithms

- **Uninformed (Blind search) search**
- The uninformed search does not contain any domain knowledge
- It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes
- **Informed search (Heuristic search)**
- Informed search algorithms use domain knowledge.
- problem information is available which can guide the search.
- Informed search is also called a Heuristic search.

Types

Uninformed search is divided into

- Breadth-first search
- Uniform cost search
- Depth-first search
- Iterative deepening depth-first search
- Bidirectional Search

Informed search

- **Greedy Search**
- A* Search

Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadth wise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there is more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Properties of Breadth-first search

- **Time Complexity:** Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

  $$\mathbf{T(b) = 1+b^2+b^3+.....+ b^d= O(b^d)}$$

- **Space Complexity:** Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.
- **Completeness:** BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.
- **Optimality:** BFS is optimal if path cost is a non-decreasing function of the depth of the node.

Breadth first search

Let *fringe* be a list containing the initial state

Loop

if *fringe* is empty return failure

Node ← remove-first (fringe)

if Node is a goal

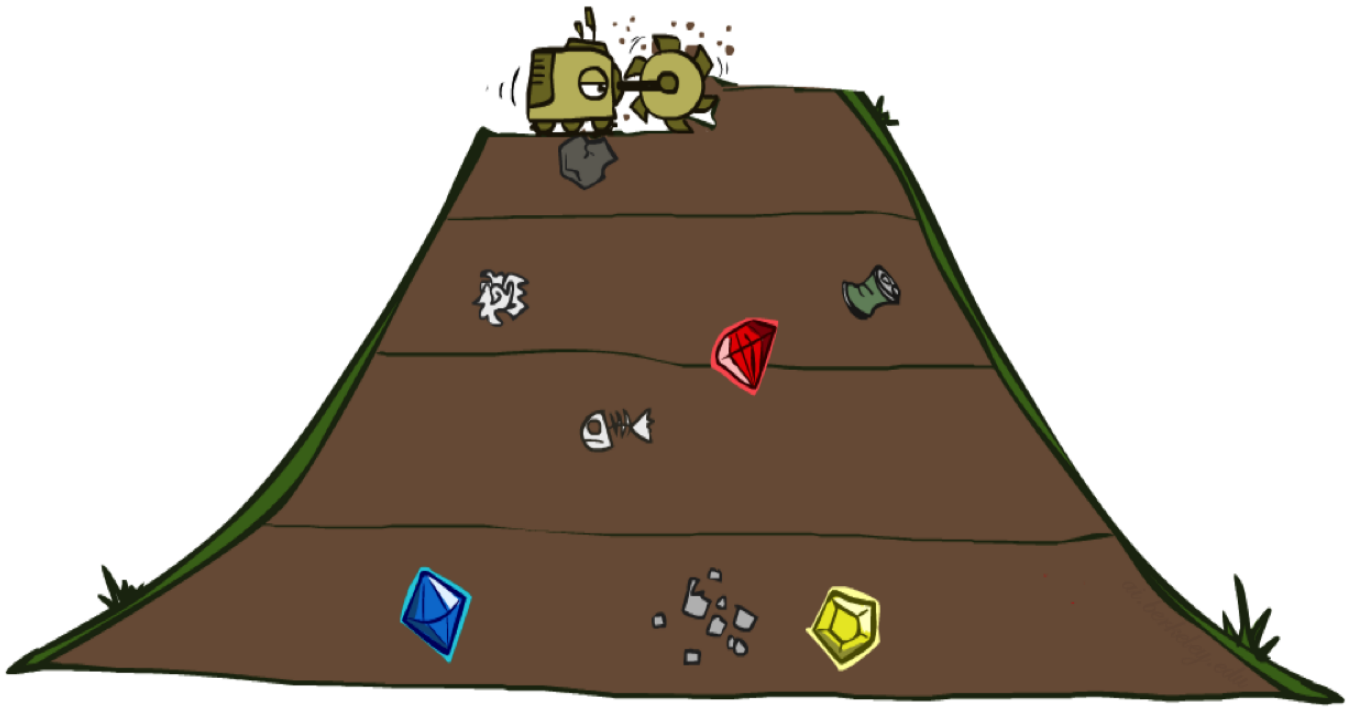
then return the path from initial state to Node

else generate all successors of Node, and

merge the newly generated nodes into *fringe*

End Loop

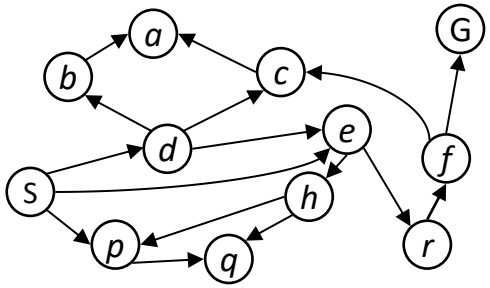
Breadth-First Search

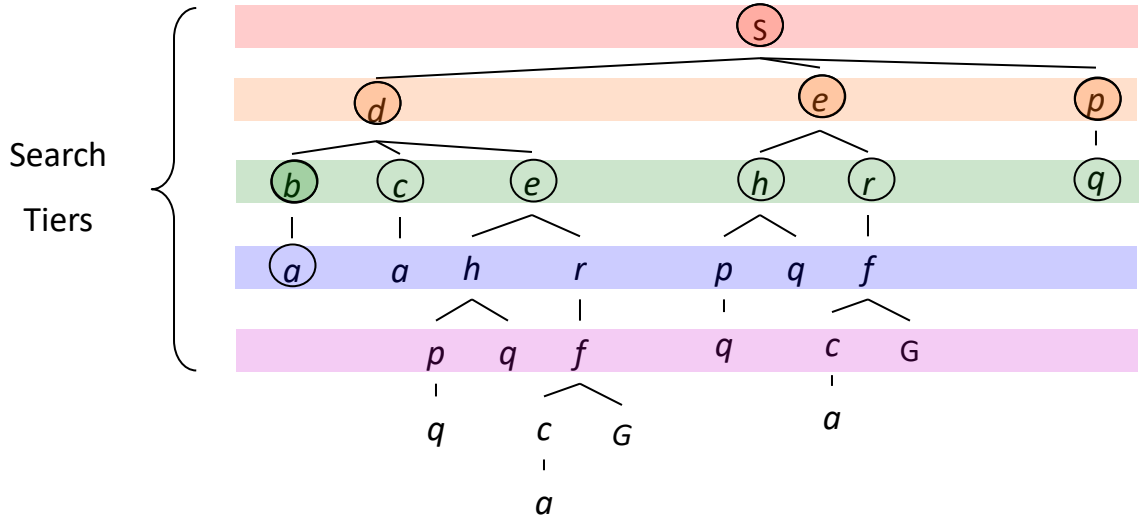


Breadth-First Search

Strategy: expand a shallowest node first

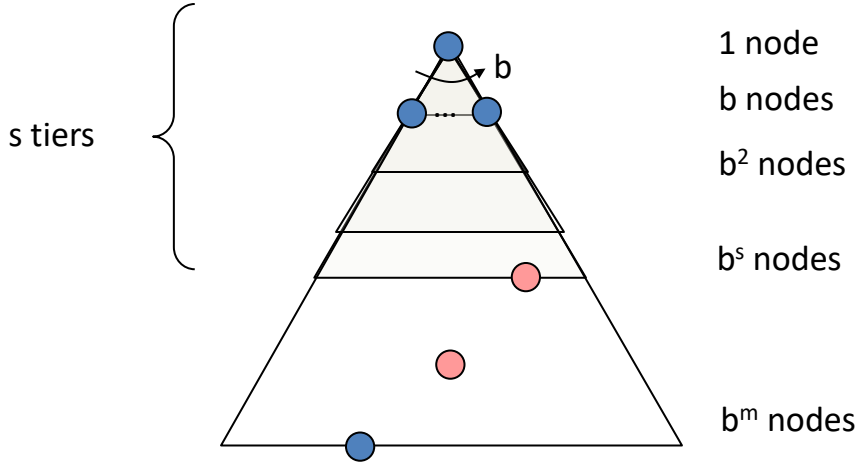
Implementation: Fringe is a FIFO queue



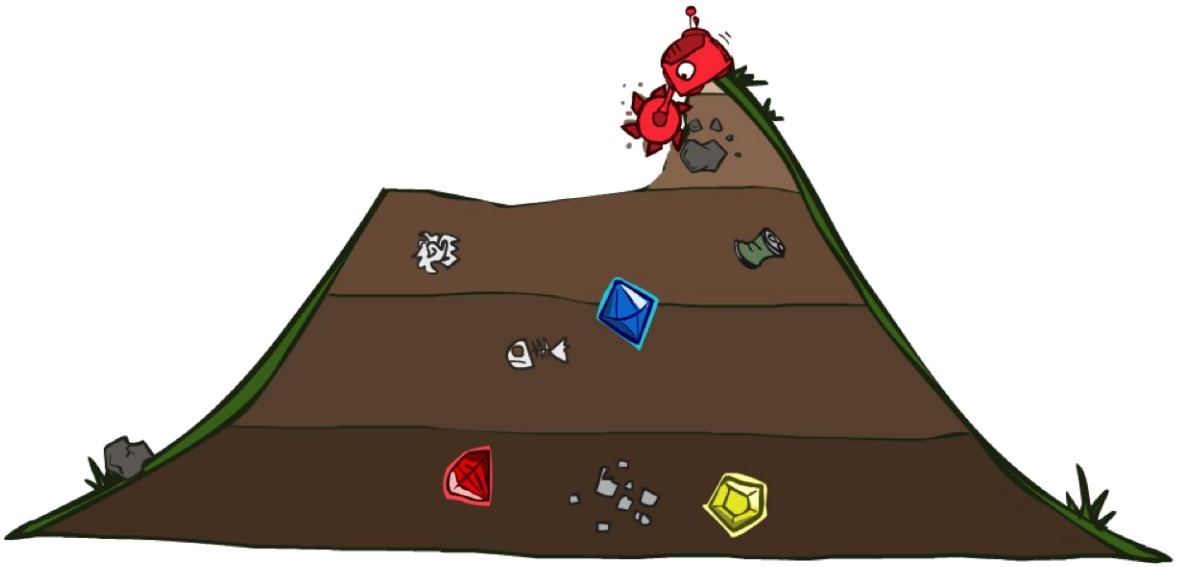


Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
  - Processes all nodes above shallowest solution
  - Let depth of shallowest solution be s
  - Search takes time $O(b^s)$
- How much space does the fringe take?
  - Has roughly the last tier, so $O(b^s)$
- Is it complete?
  - s must be finite if a solution exists, so yes!
- Is it optimal?
  - Only if costs are all 1 (more on costs later)



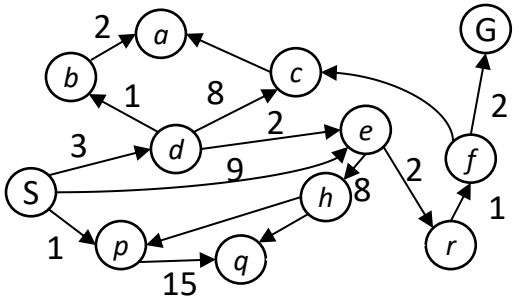
Uniform Cost Search

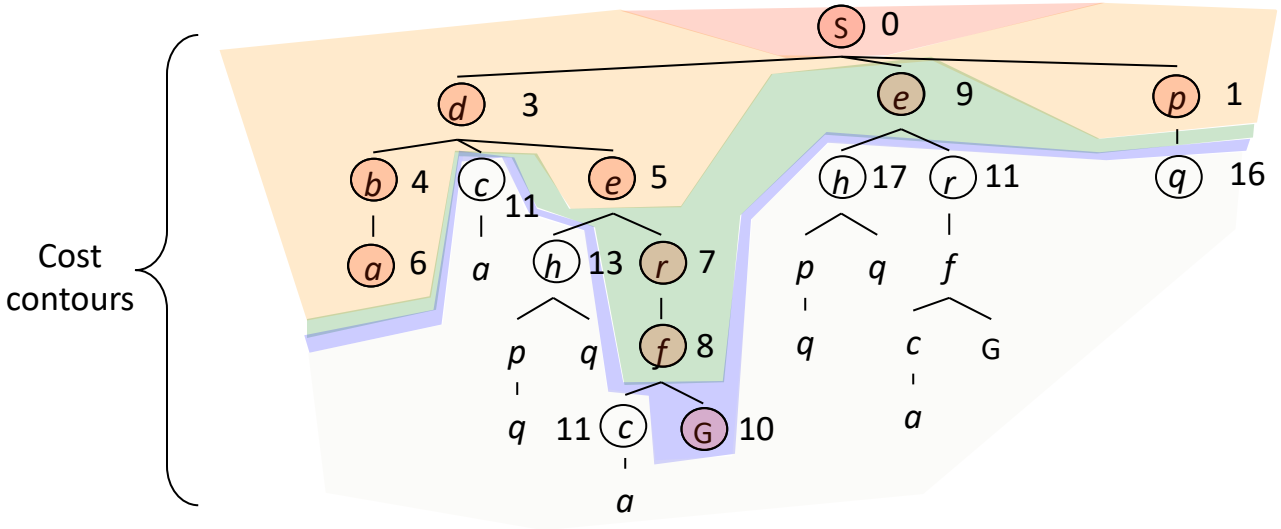


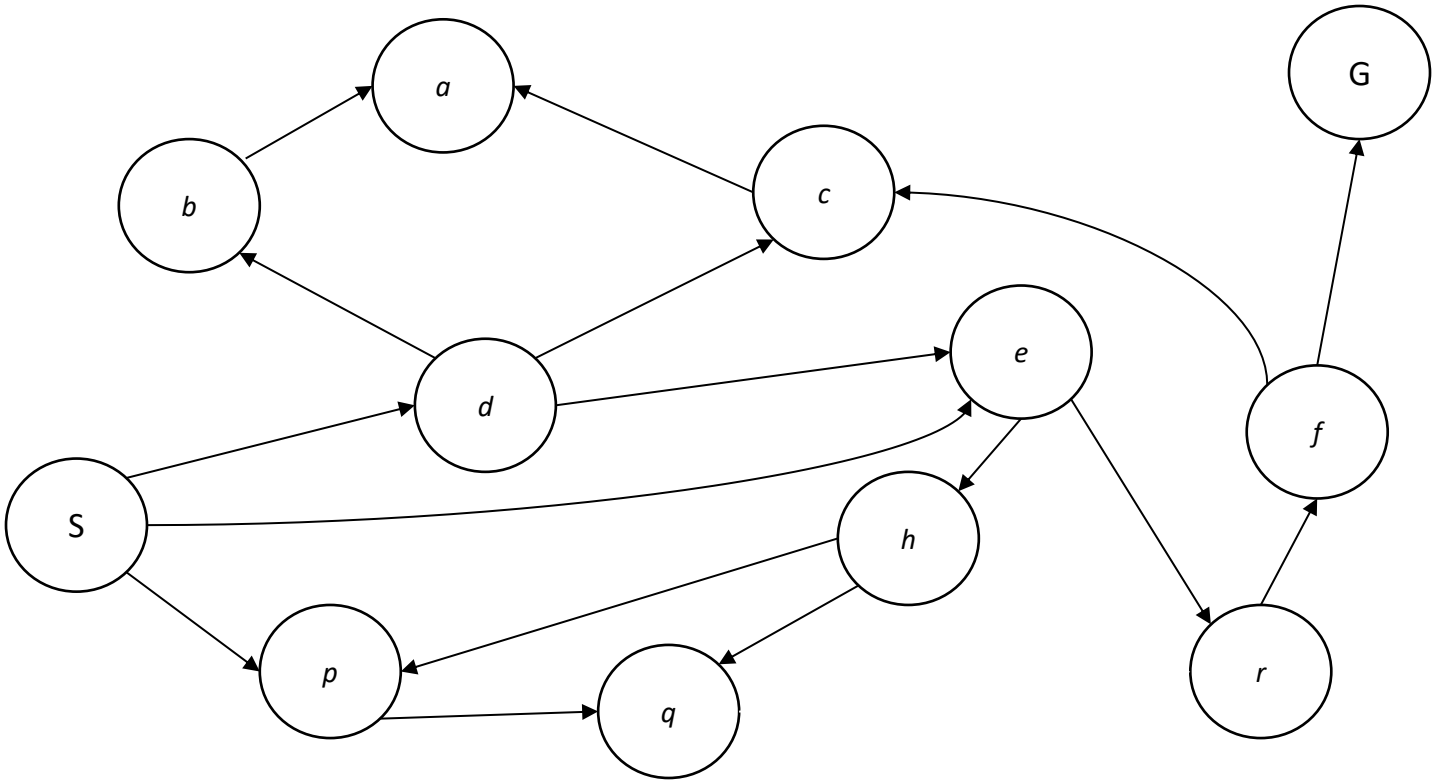
Uniform Cost Search

Strategy: expand a cheapest node first:

Fringe is a priority queue (priority: cumulative cost)

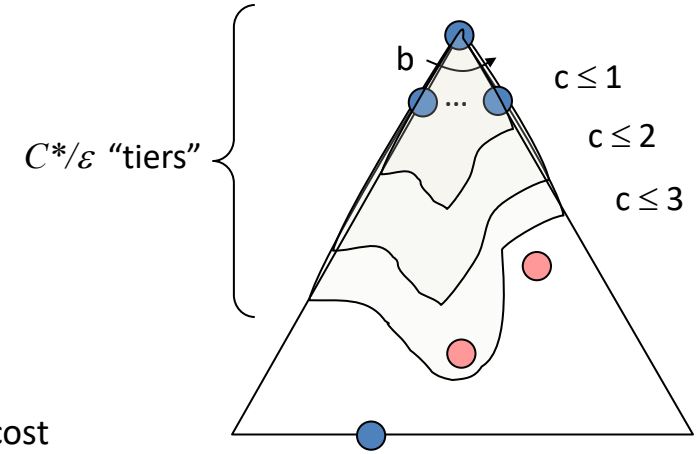





G
a
b
c
e
d
f
S
h
p
r
q

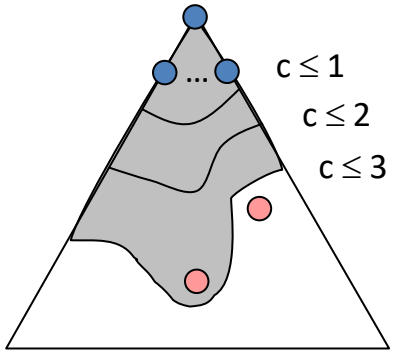
Uniform Cost Search (UCS) Properties

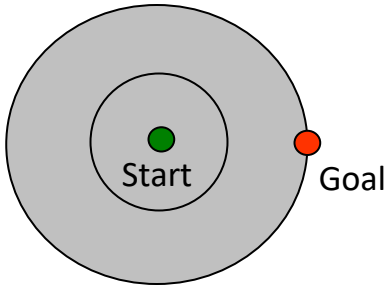
- What nodes does UCS expand?
  - Processes all nodes with cost less than cheapest solution!
  - If that solution costs C^* and arcs cost at least ε, then the "effective depth" is roughly C^*/ε
  - Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)
- How much space does the fringe take?
  - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
  - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
  - Yes! (Proof next lecture via A*)



Uniform Cost Issues

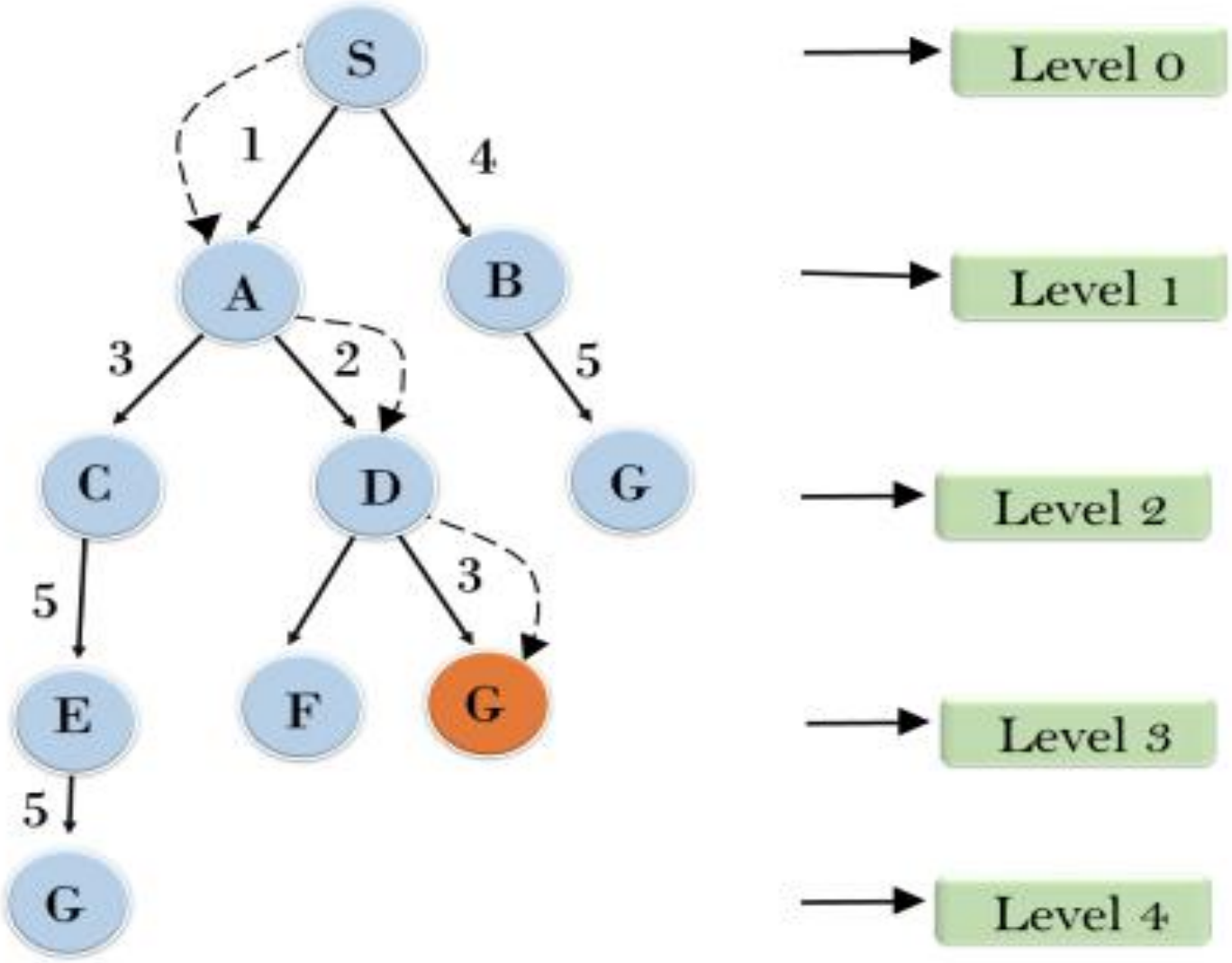
- Remember: UCS explores increasing cost contours
- The good: UCS is complete and optimal!
- The bad:
  - Explores options in every “direction”
  - No information about goal location
- We’ll fix that soon!





[Demo: empty grid UCS (L2D5)]
[Demo: maze with deep/shallow water DFS/BFS/UCS (L2D7)]

Uniform Cost Search



- **Completeness:**
- Uniform-cost search is complete, such as if there is a solution, UCS will find it.
- **Time Complexity:**
- Let C* **is Cost of the optimal solution**, and **ε** is each step to get closer to the goal node. Then the number of steps is = C*/ε+1. Here we have taken +1, as we start from state 0 and end to C*/ε.
- Hence, the worst-case time complexity of Uniform-cost search is$\mathbf{O(b^{1+[C^*/\varepsilon]})/}$.
- **Space Complexity:**
- The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $\mathbf{O(b^{1+[C^*/\varepsilon]})}$.
- **Optimal:**
- Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

Depth-first Search

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Example

- In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:
- Root node--->Left node ----> right node.
- It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

Depth-first Search

- **Completeness:** DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.
- **Time Complexity:** Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$\mathbf{T(n)= 1+ n^2+ n^3 +.....+ n^m=O(n^m)}$$

- **Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)**
- **Space Complexity:** DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.
- **Optimal:** DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

Depth first search

Let *fringe* be a list containing the initial state

Loop

if *fringe* is empty return failure

Node ← remove-first (*fringe*)

if Node is a goal

then return the path from initial state to Node

else generate all successors of Node, and

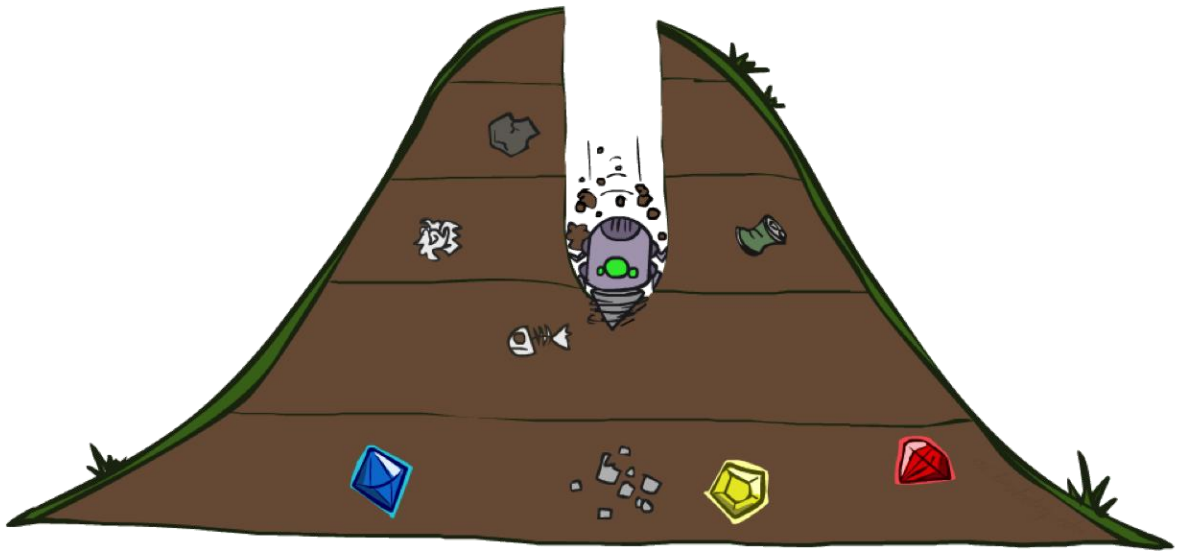
merge the newly generated nodes into *fringe*

End Loop

Depth-first Search

- **Advantage:**
- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).
- **Disadvantage:**
- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

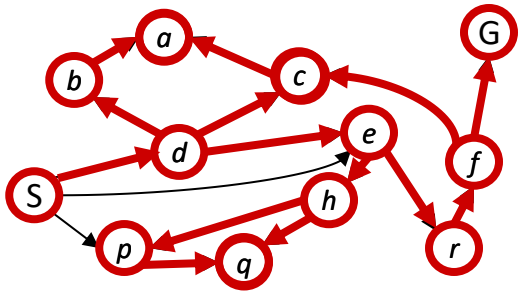
Depth-First Search

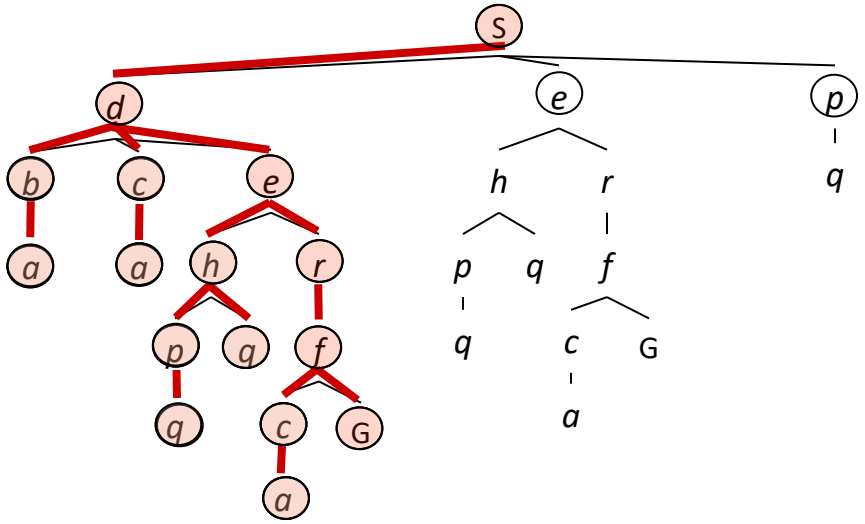


Depth-First Search

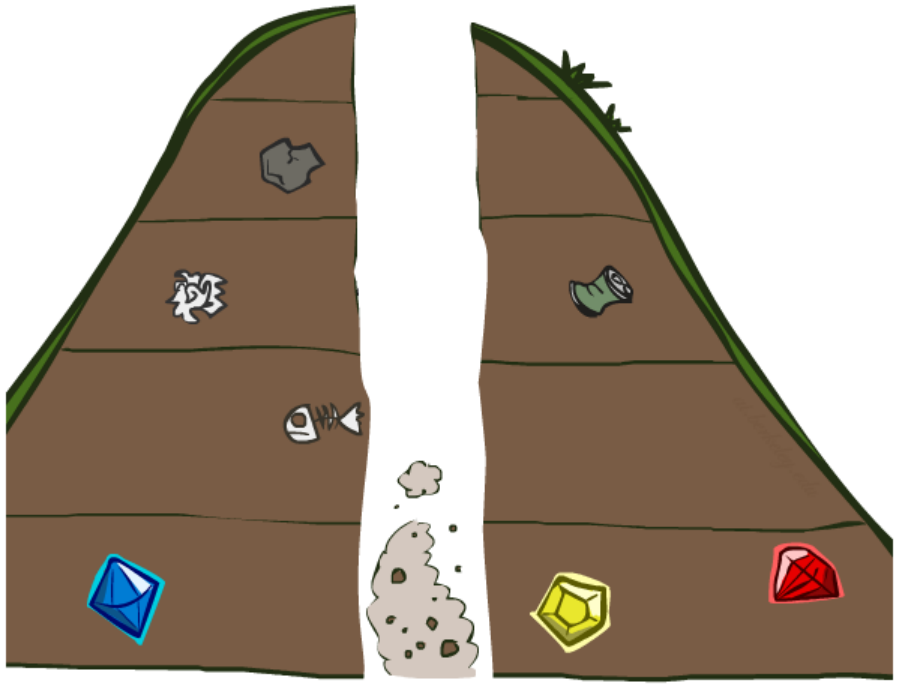
Strategy: expand a deepest node first

Implementation: Fringe is a LIFO stack





Search Algorithm Properties

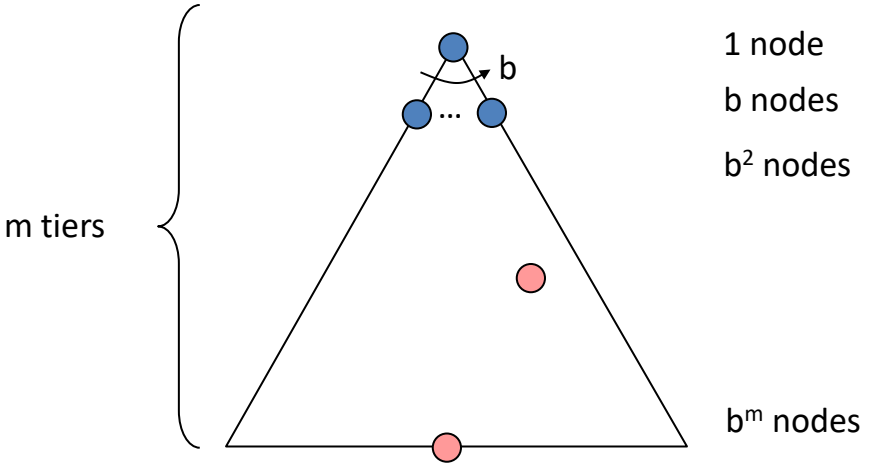


Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?

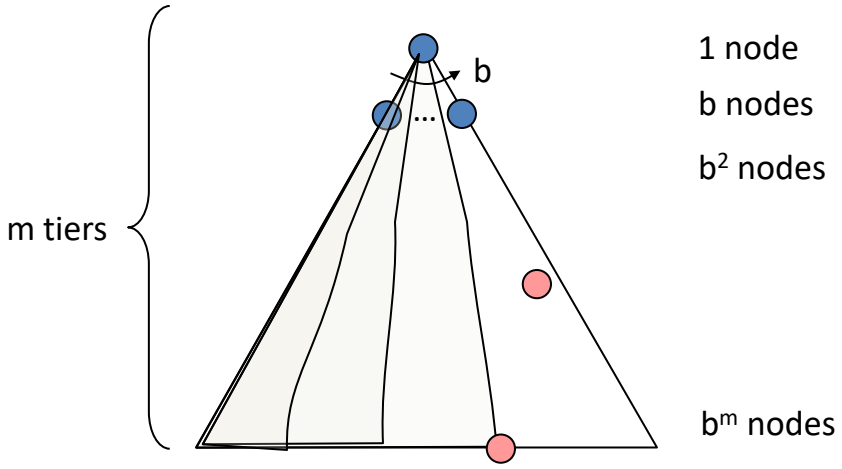
- factors of search tree:
  - b is the branching factor
  - m is the maximum depth
  - solutions at various depths

- Number of nodes in entire tree?
  - $1 + b + b^2 + \ldots. b^m = O(b^m)$



Depth-First Search (DFS) Properties

- What nodes DFS expand?
  - Some left prefix of the tree.
  - Could process the whole tree!
  - If m is finite, takes time $O(b^m)$
- How much space does the fringe take?
  - Only has siblings on path to root, so O(bm)
- Is it complete?
  - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
  - No, it finds the "leftmost" solution, regardless of depth or cost



Depth-Limited Search Algorithm:

- A depth-limited search algorithm is similar to depth-first search with a predetermined limit.
- Depth-limited search can solve the drawback of the infinite path in the Depth-first search.
- In this algorithm, the node at the depth limit will treat as it has no successor nodes further.
- Depth-limited search can be terminated with two Conditions of failure:
- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.

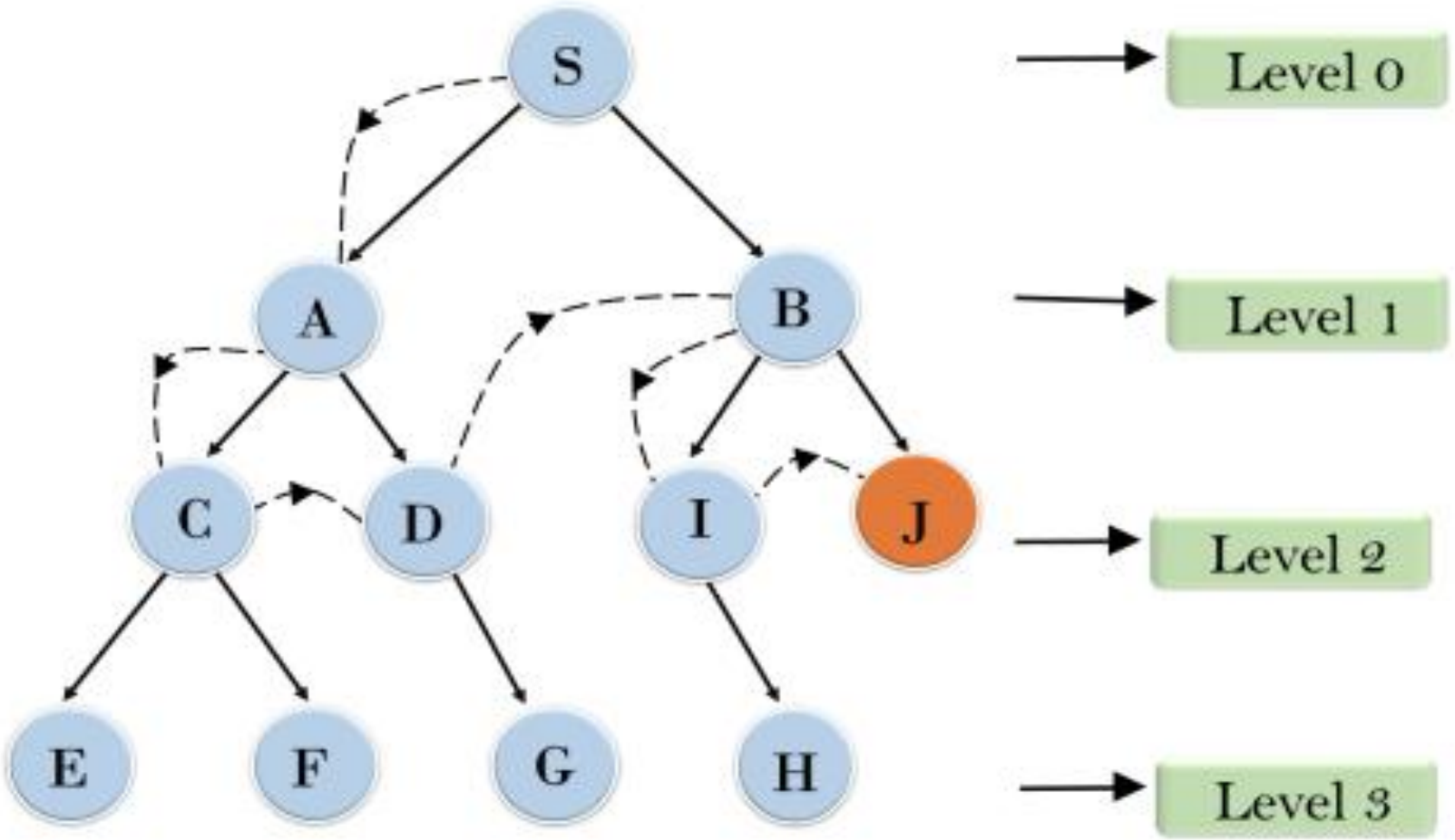
Depth limited search (limit)

```
Let fringe be a list containing the initial state
Loop
        if fringe is empty return failure
        Node ← remove-first (fringe)
        if Node is a goal
            then return the path from initial state to Node
        else if depth of Node = limit return cutoff
        else add generated nodes to the front of fringe
End Loop
```

Depth-Limited Search Algorithm:

- **Advantages:**
- Depth-limited search is Memory efficient.
- **Disadvantages:**
- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

Depth Limited Search



Depth-Limited Search Algorithm:

- **Completeness:** DLS search algorithm is complete if the solution is above the depth-limit.
- **Time Complexity:** Time complexity of DLS algorithm is **$O(b^{\ell})$**.
- **Space Complexity:** Space complexity of DLS algorithm is **$O(b \times \ell)$**.
- **Optimal:** Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$.

Informed Search Algorithms

- Informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node.
- This knowledge help agents to explore less to the search space and find more efficiently the goal node.
- Informed search algorithm uses the idea of heuristic function.
- **Heuristic function** :It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.
- The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time.

Example of Heuristic Function

- A heuristic function at a node n is an estimate of the optimum cost from the current node to a goal. Denoted by *h(n)*
  h(n) = estimated cost of the cheapest path from node n to a goal node
- Example
  - Want path from Kolkata to Guwahati
  - Heuristic for Guwahati may be straight-line distance between Kokata and Guwahati
  - *h(Kolkata)* = euclideanDistance(Kolkata, Guwahati)

Admissibility of the heuristic:-

- Admissibility of the heuristic function is given as:

$$h(n) <= h^*(n)$$

- Here h(n) is heuristic cost, and h*(n) is the estimated cost.
- Hence heuristic cost should be less than or equal to the estimated cost.

Heuristic search

- Heuristic search is the simplest form of heuristic search algorithms.
- It expands nodes based on their heuristic value h(n).
- It maintains two lists, OPEN and CLOSED list.
- In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.
- On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list.
- The algorithm continues unit a goal state is found.

Informed Search Algorithms

- **Best First Search Algorithm(Greedy search)**
- **A* Search Algorithm**

Best First Search Algorithm(Greedy search)

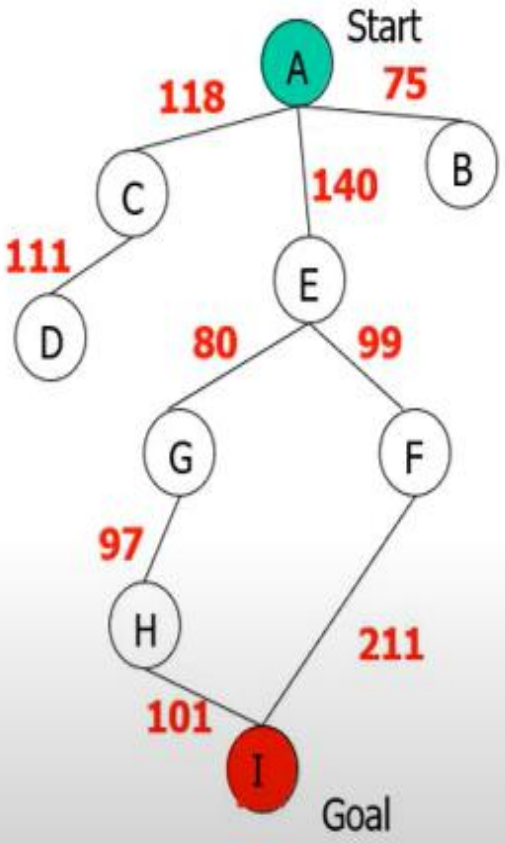
- Greedy best-first search algorithm always selects the path which appears best at that moment.
- It is the combination of depth-first search and breadth-first search algorithms.
- It uses the heuristic function and search.
- Best-first search allows us to take the advantages of both algorithms.
- At each step, we can choose the most promising node.
- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function,

Greedy best-first search

- The evaluation function is f(n) = h(n)
- Were, h(n)= estimated cost from node n to the goal.

- Greedy search ignores the cost of the path that has already been traversed to reach n
- Therefore, the solution given is not necessarily optimal

Greedy best-first search...

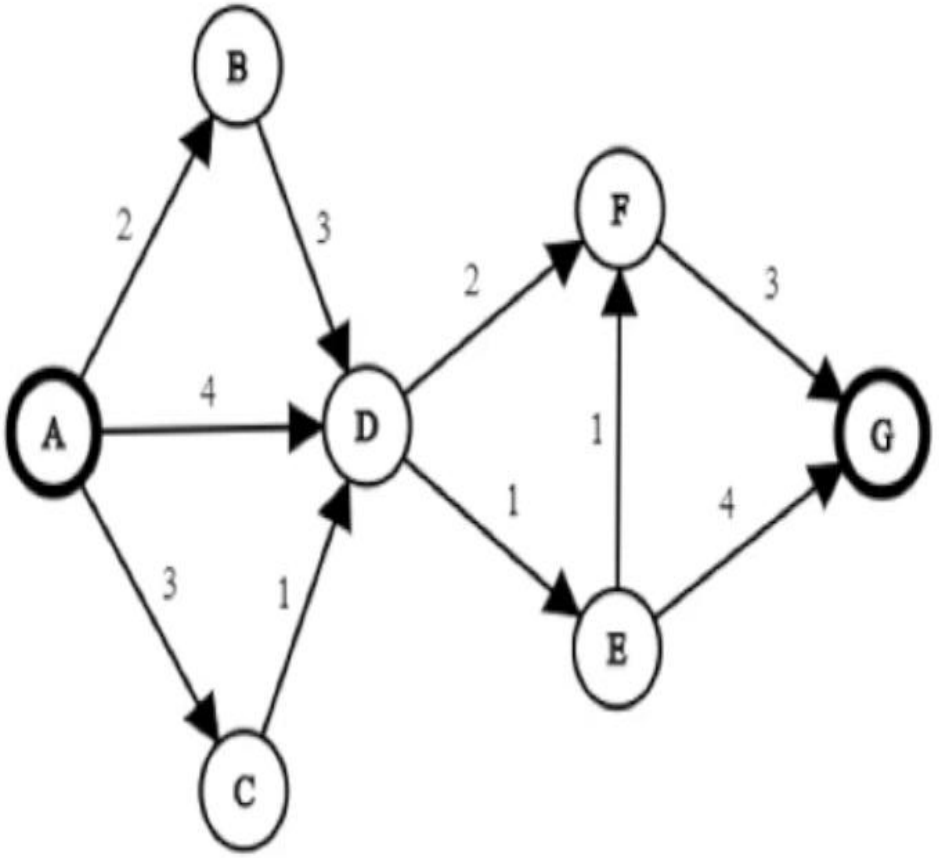


State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

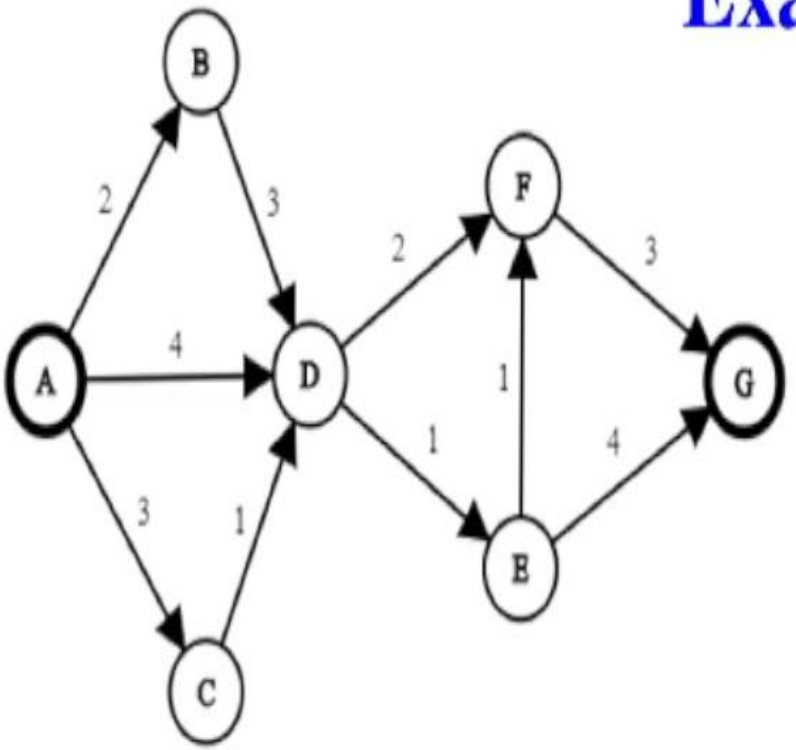
***f(n) = h(n)* = straight-line distance heuristic**

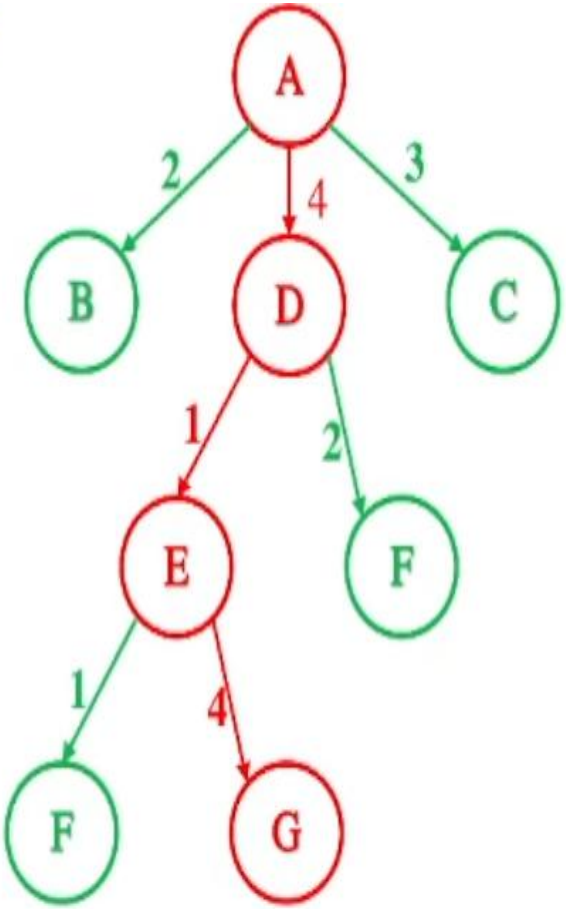
Example

- Consider the following directed graph, having A as the starting node and G as the goal node, with edge costs as mentioned, and the heuristic values for the nodes are given as - {h(A)=7, h(B)=6, h(C)=5, h(D)=4, h(E)=3, h(F)=3, h(G)=0}.



Example





h(A)=7, h(B)=6, h(C)=5, h(D)=4,

h(E)=3, h(F)=3, h(G)=0.

Path : A – D – E - G

Path Cost :4 + 1+ 4 = 9

Best first search algorithm:

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
- **Step 4:** Expand the node n, and generate the successors of node n.
- **Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Best First Search

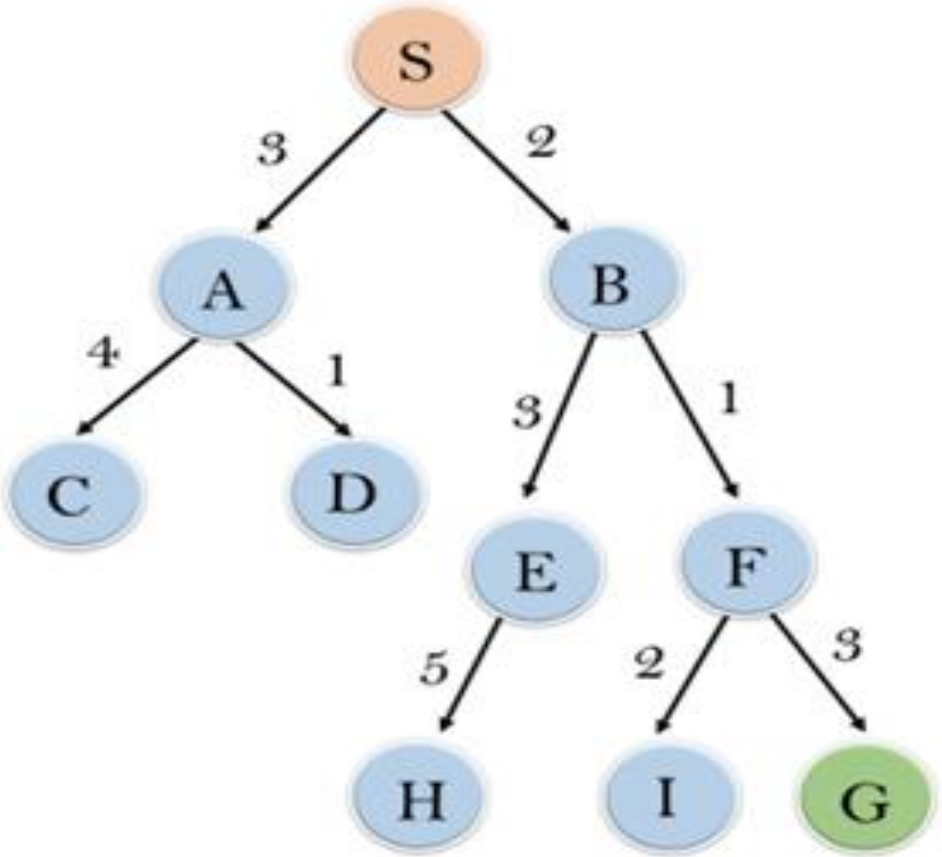
Let *fringe* be a *priority queue* containing the initial state

Loop

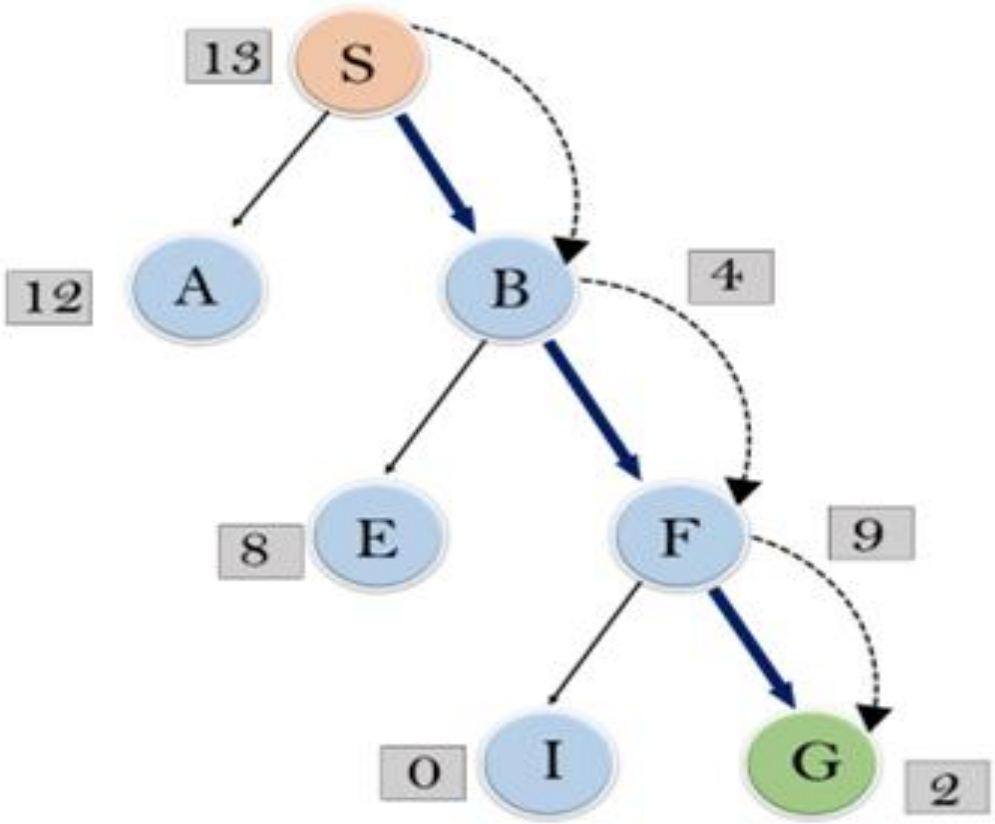
if *fringe* is empty return *failure*
Node ← remove-first (*fringe*)
if Node is a *goal*
then return the path from initial state to Node
else generate all successors of Node, and
put the newly generated nodes into *fringe*
according to their f values

End Loop

- Advantages:
- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.
- Disadvantages:
- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.



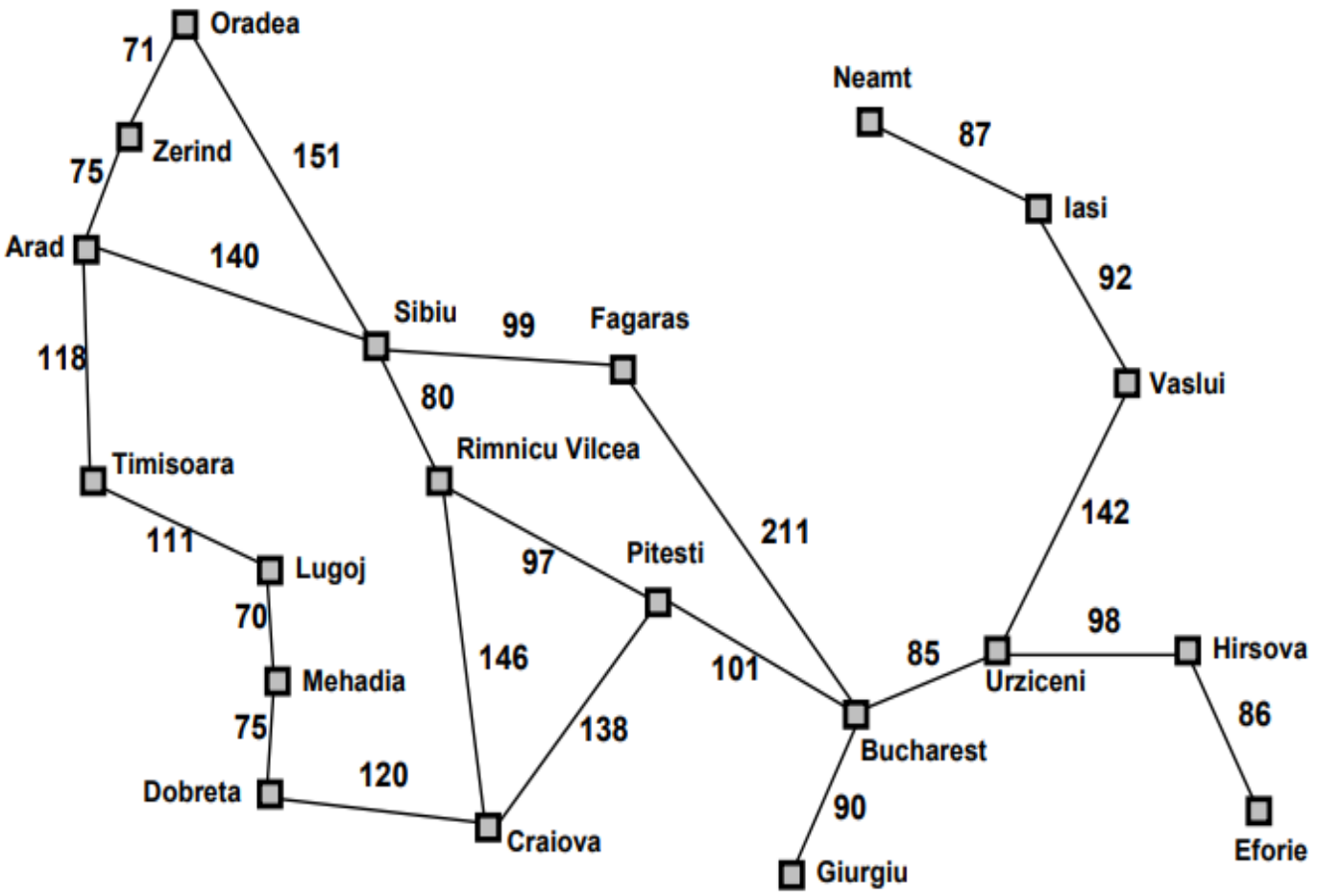
node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0


13
S
12
A
B
4
8
E
F
9
0
I
G
2

- **Expand the nodes of S and put in the CLOSED list**
- **Initialization:** Open [A, B], Closed [S]
- **Iteration 1:** Open [A], Closed [S, B]
- **Iteration 2:** Open [E, F, A], Closed [S, B]
  : Open [E, A], Closed [S, B, F]
- **Iteration 3:** Open [I, G, E, A], Closed [S, B, F]
  : Open [I, E, A], Closed [S, B, F, G]
- Hence the final solution path will be:
- **S----> B----->F----> G**

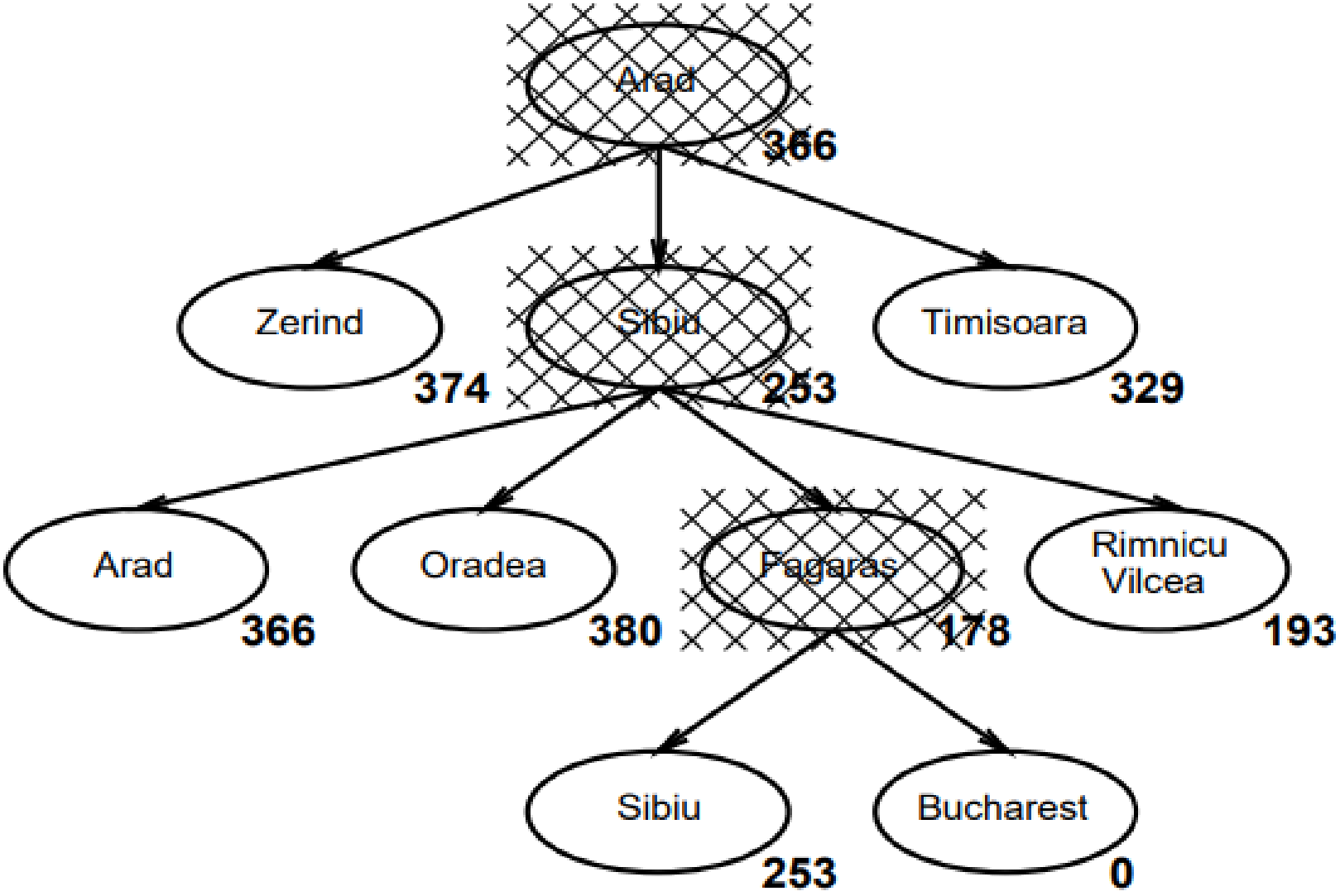
- **Time Complexity:** The worst case time complexity of Greedy best first search is $O(b^m)$.
- **Space Complexity:** The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.
- **Complete:** Greedy best-first search is also incomplete, even if the given state space is finite.
- **Optimal:** Greedy best first search algorithm is not optimal.

Romania with step costs in km



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Activate
Go to Settir


Arad
366
Zerind
374
Sibiu
253
Timisoara
329
Arad
366
Oradea
380
Pagaras
178
Rimnicu
Vilcea
193
Sibiu
253
Bucharest
0

A* Search Algorithm:

- A* search is the most commonly known form of best-first search.
- It uses heuristic function h(n), and cost to reach the node n from the start state g(n).
- It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently
- This search algorithm expands less search tree and provides optimal result faster.
- A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).

A* Search Algorithm:

- **Step1:** Place the starting node in the OPEN list.
- **Step 2:** Check if the OPEN list is empty or not, if the list is empty then return failure and stops.
- **Step 3:** Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise
- **Step 4:** Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.
- **Step 5:** Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.
- **Step 6:** Return to **Step 2**.

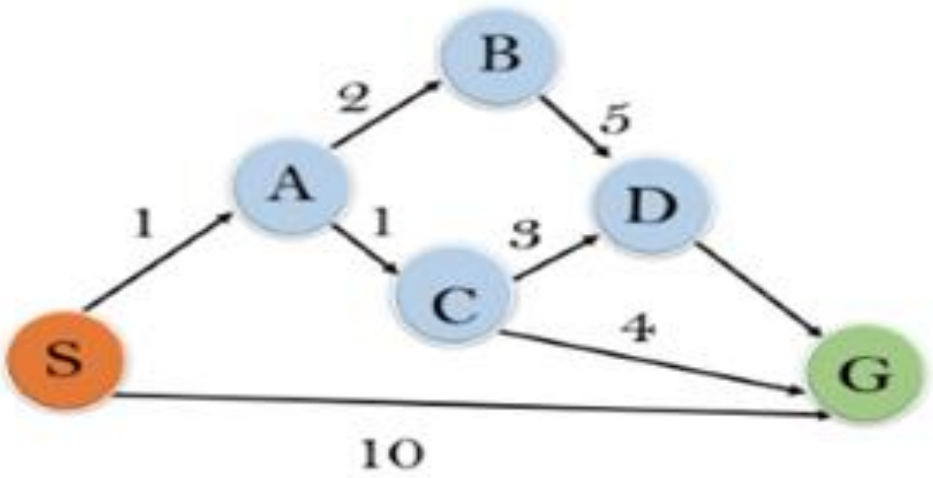
Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

Disadvantages:

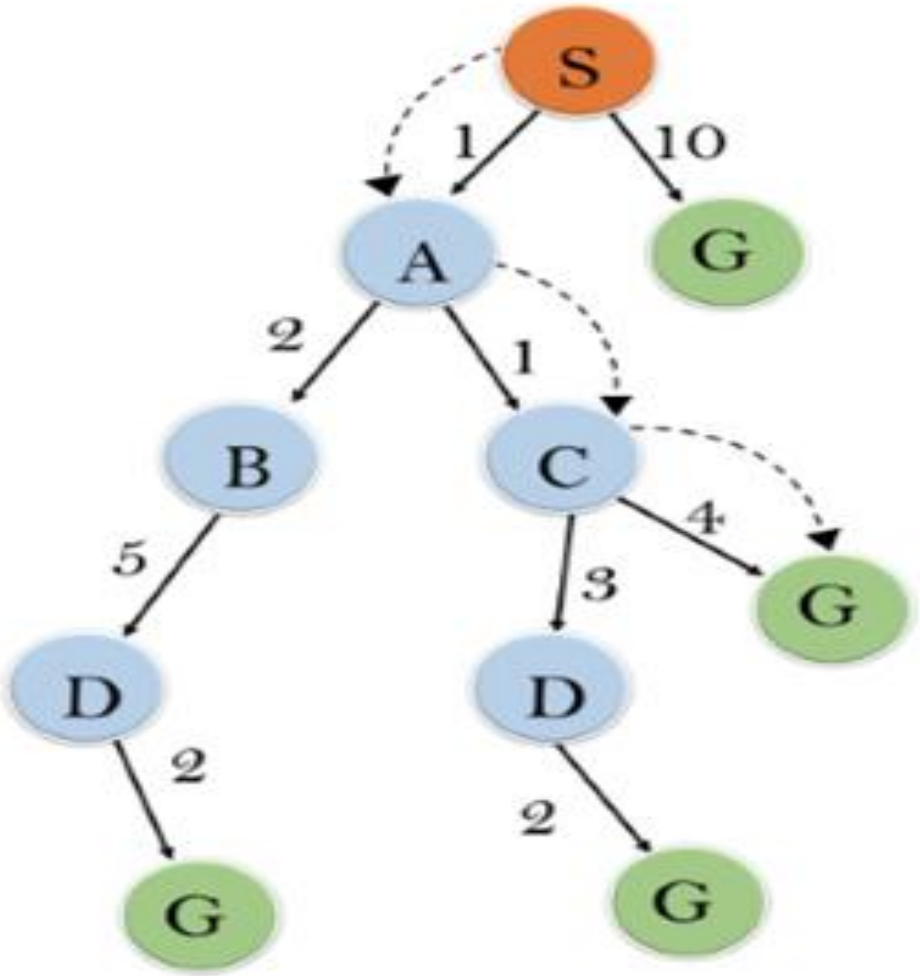
- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

A* Search Algorithm:



A* Search Algorithm:

- **Initialization:** {(S, 5)}
- **Iteration1:** {(S--> A, 4), (S-->G, 10)}
- **Iteration2:** {(S--> A-->C, 4), (S--> A-->B, 7), (S-->G, 10)}
- **Iteration3:** {(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S-->G, 10)}
- **Iteration 4** will give the final result, as **S--->A--->C--->G** it provides the optimal path with cost 6.

A* Search Algorithm:

- **Complete:** A* algorithm is complete as long as:
- Branching factor is finite.
- Cost at every action is fixed.
- **Optimal:** A* search algorithm is optimal if it follows below two conditions:
- **Admissible:** the first condition requires for optimality is that h(n) should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.
- If the heuristic function is admissible, then A* tree search will always find the least cost path.
- **Time Complexity:** The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d. So the time complexity is O(b^d), where b is the branching factor.
- **Space Complexity:** The space complexity of A* search algorithm is **O(b^d)**

Algorithm A*

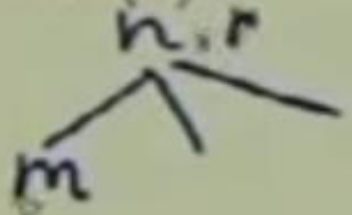
OPEN = nodes on frontier. CLOSED = expanded nodes.

OPEN = {<s, nil>}

while OPEN is not empty

remove from OPEN the node <n,p> with minimum *f(n)*

place <*n,p*> on CLOSED



if *n* is a goal node, return success (path *p*)

for each edge connecting *n* & *m* with cost c

if <m, q> is on CLOSED and *{p|e}* is cheaper than *q*

then remove *n* from CLOSED, put <*m,{p|e}*> on OPEN

else if <*m,q*> is on OPEN and *{p|e}* is cheaper than *q*

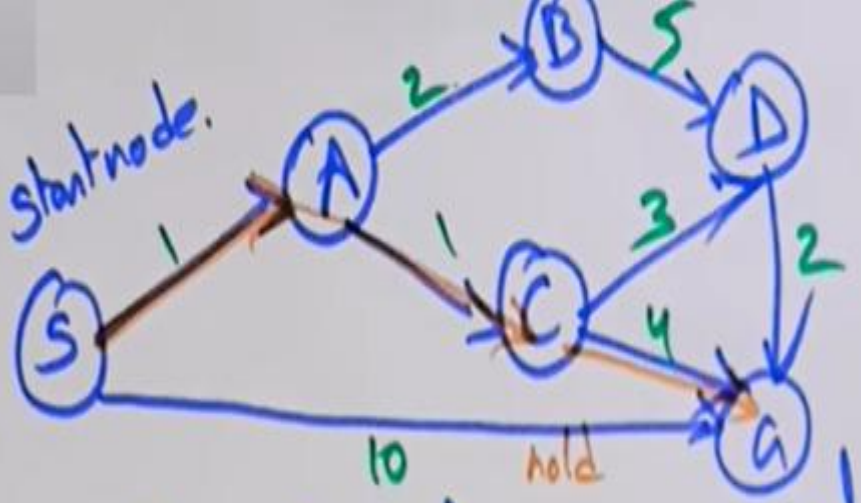
then replace *q* with *{p|e}*

else if *m* is not on OPEN put <*m,{p|e}*> on OPEN

return *failure*

Example 1

State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0



① $S \to A \Rightarrow f(n) = g(n) + h(n)$
$= 1 + 3 = 4$ ✓

$S \to A \to C \to$

$S \to G = f(n) = 10 + 0 = 10$ hold ✗

② $S \to A \to B \Rightarrow f(n) = 3 + 4 = 7$ hold ✗

$S \to A \to C \Rightarrow f(n) = 2 + 2 = 4$ ✓

③ $S \to A \to C \to D \Rightarrow f(n) = 5 + 6 = 11$ hold ✗

$S \to A \to C \to G \Rightarrow f(n) = 6 + 0 = 6$ ✓

Hill Climbing Algorithm

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem.
- It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.

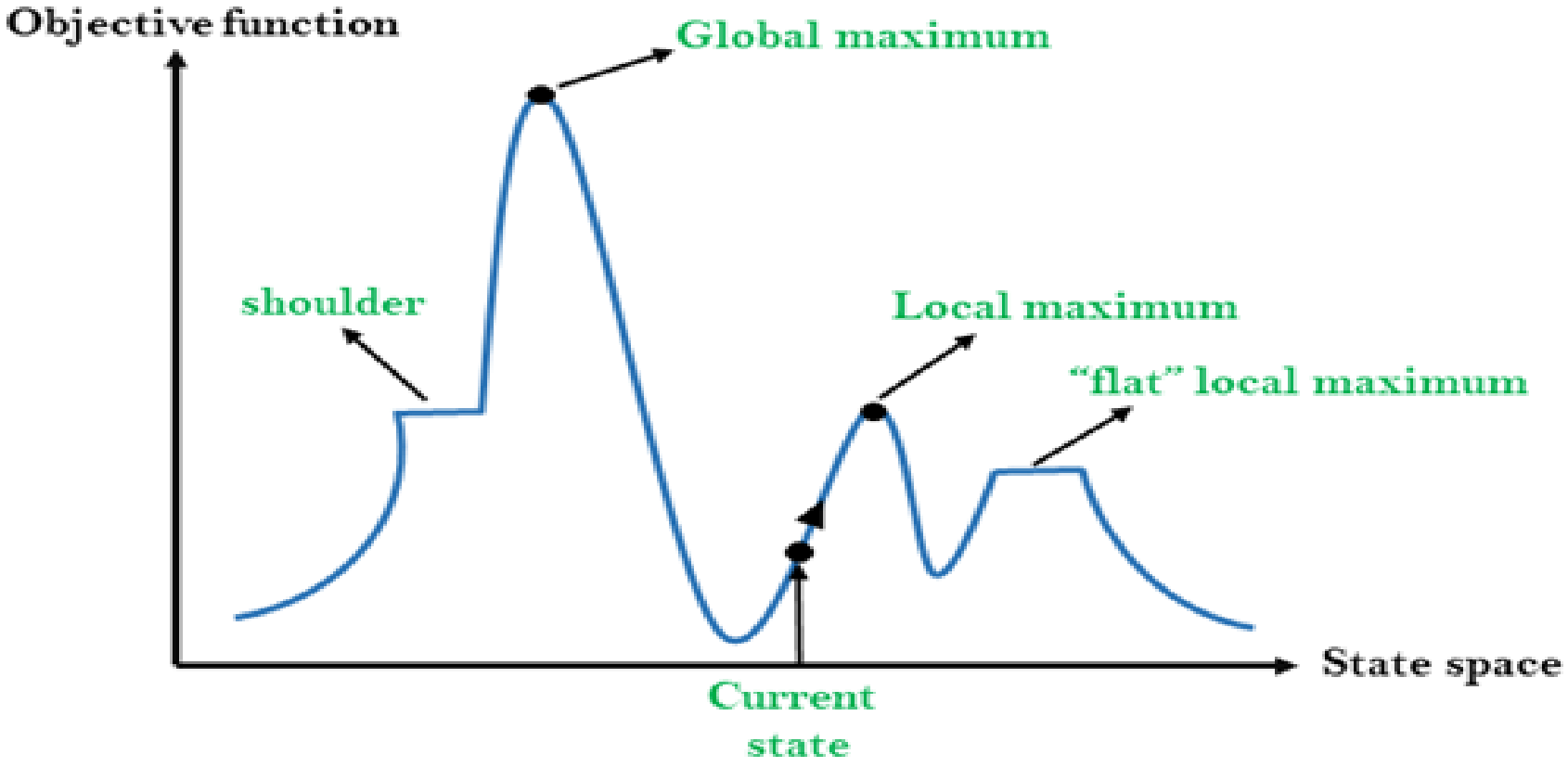
Features of Hill Climbing:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

Different regions in the state space landscape

- **Local Maximum:** Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.
- **Global Maximum:** Global maximum is the best possible state of state space landscape. It has the highest value of objective function.
- **Current state:** It is a state in a landscape diagram where an agent is currently present.
- **Flat local maximum:** It is a flat space in the landscape where all the neighbor states of current states have the same value.
- **Shoulder:** It is a plateau region which has an uphill edge.

State-space Diagram



The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

Types of Hill Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

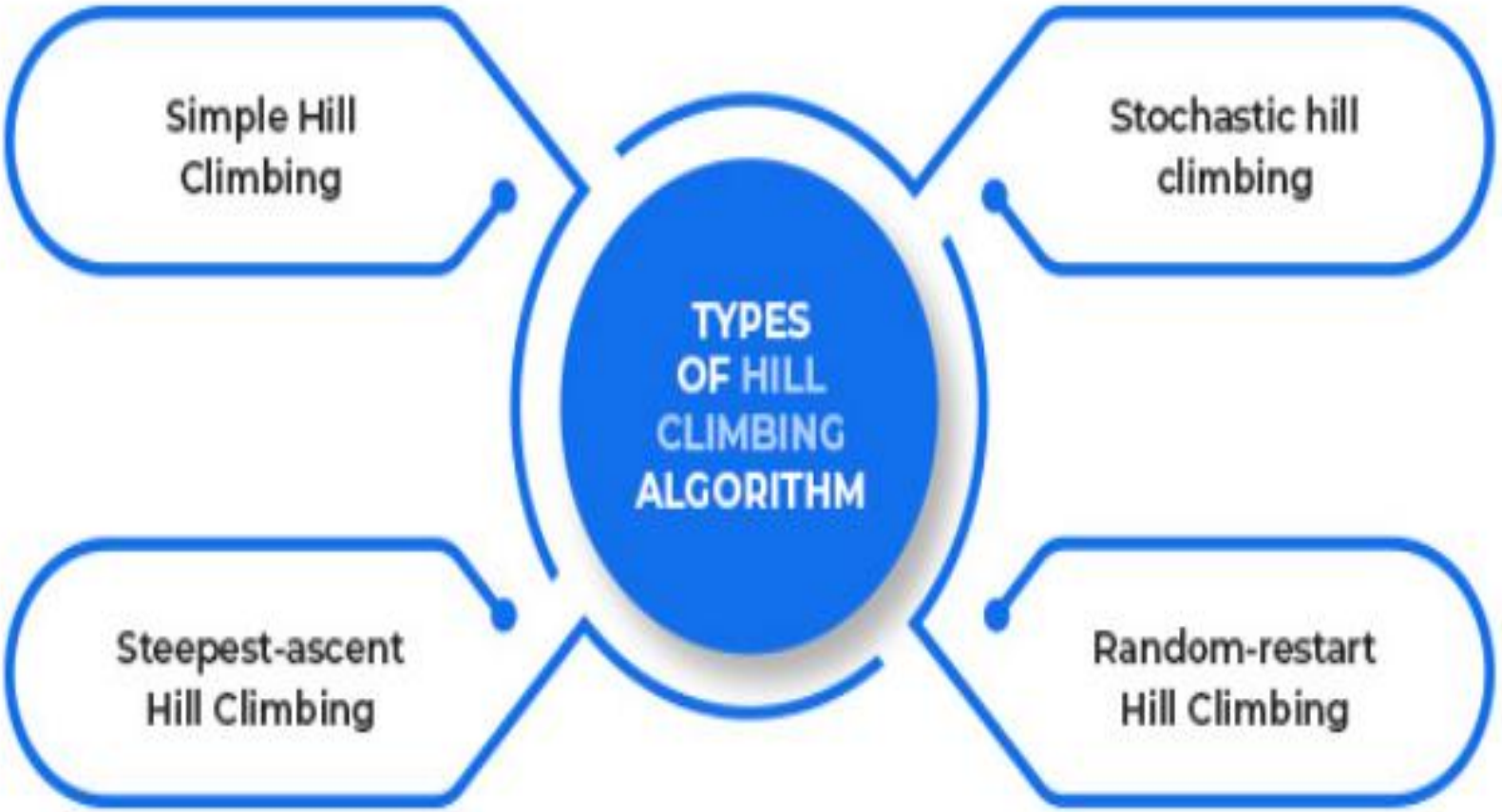
1. Simple Hill Climbing:

- Simple hill climbing is the simplest way to implement a hill climbing algorithm.
- **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state**.
- It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state.

This algorithm has the following features:

- Less time consuming
- Less optimal solution and the solution is not guaranteed

Types of Hill Climbing Algorithm:



```
function HILL-CLIMBING( problem) returns a solution state
    inputs: problem, a problem
    local variables: current, a node
                     next, a node

    current ← MAKE-NODE(INITIAL-STATE[problem])
    loop do
        next ← a highest-valued successor of current
        if VALUE[next] < VALUE[current] then return current
        current ← next
    end
```

Algorithm for Simple Hill Climbing:

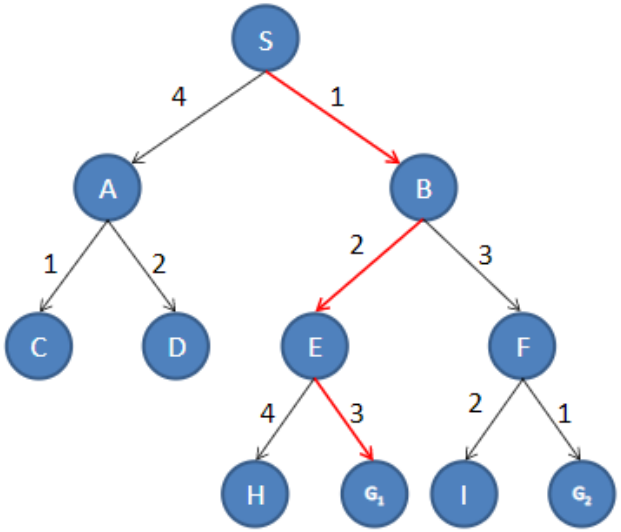
- **Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check new state:
  - If it is goal state, then return success and quit.
  - Else if it is better than the current state then assign new state as a current state.
  - Else if not better than the current state, then return to step2.
- **Step 5:** Exit.

EXAMPLE on HILL-CLIMBING

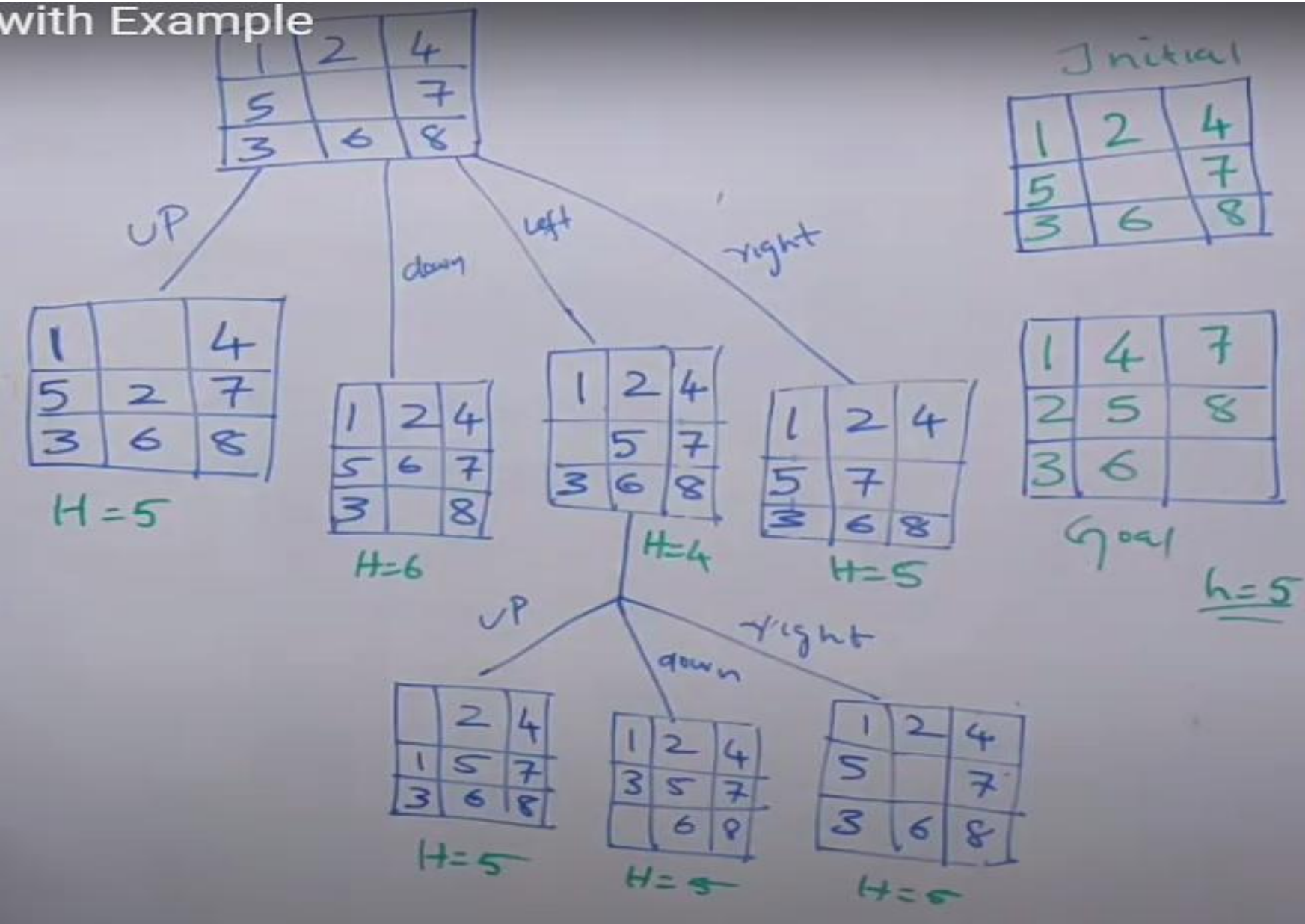
S → B → E → G1

SEARCH PATH = [S_0, B_1, E_2, $G1_3$]

Cost =6=1+2+3



Example


with Example
Initial
UP
down
left
right
H=5
H=6
H=4
H=5
Goal
h=5
UP
down
right
H=5
H=5
H=5

2. Steepest-Ascent hill climbing:

- The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors

Algorithm for Steepest-Ascent hill climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
- **Step 2:** Loop until a solution is found or the current state does not change.
  - Let SUCC be a state such that any successor of the current state will be better than it.
  - For each operator that applies to the current state:
    - Apply the new operator and generate a new state.
    - Evaluate the new state.
    - If it is goal state, then return it and quit, else compare it to the SUCC.
    - If it is better than SUCC, then set new state as SUCC.
    - If the SUCC is better than the current state, then set current state to SUCC.
- **Step 5:** Exit.

3. Stochastic hill climbing:

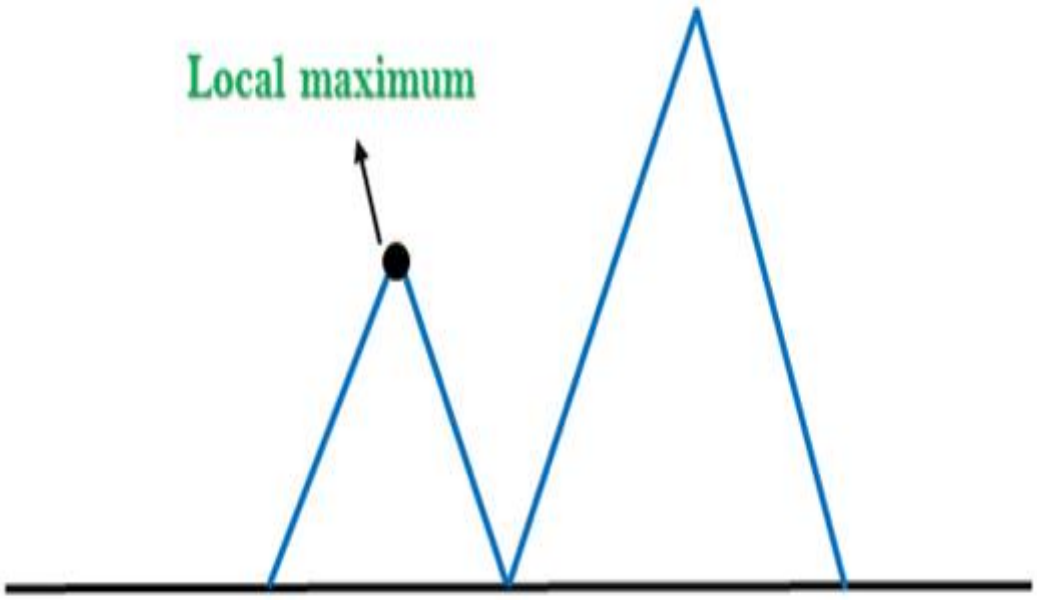
- Stochastic hill climbing does not examine for all its neighbor before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

Problems in Hill Climbing Algorithm:

- **1. Local Maximum:** A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.

- **Solution:** Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.

Local Maximum

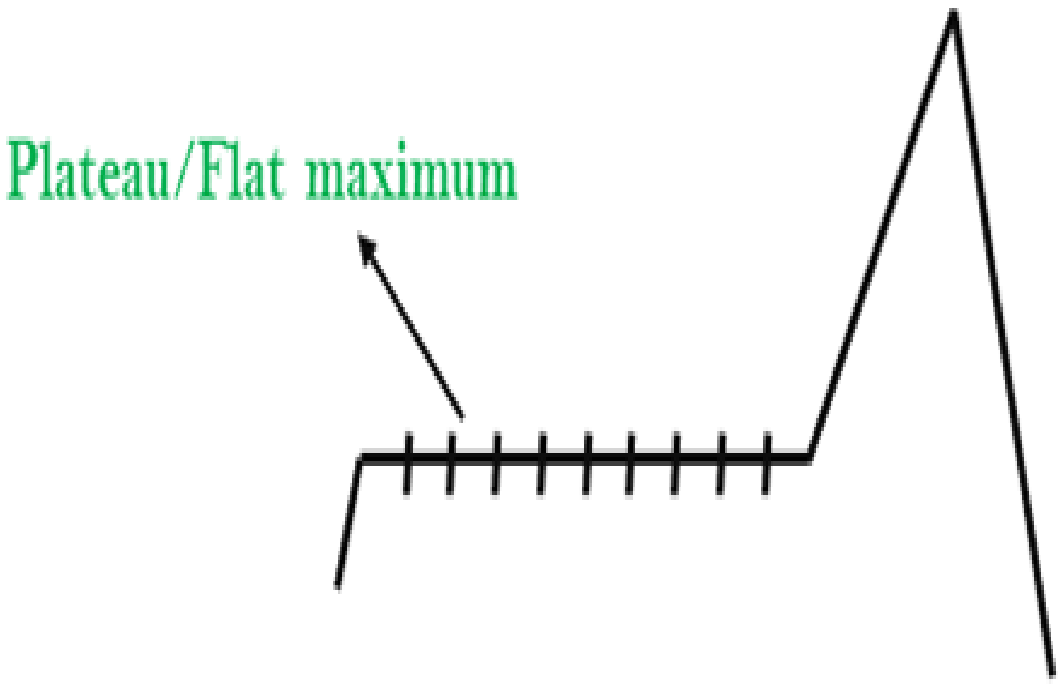


Problems in Hill Climbing Algorithm:

- **2. Plateau:** A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.

- **Solution:** The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find non-plateau region.

Plateau:

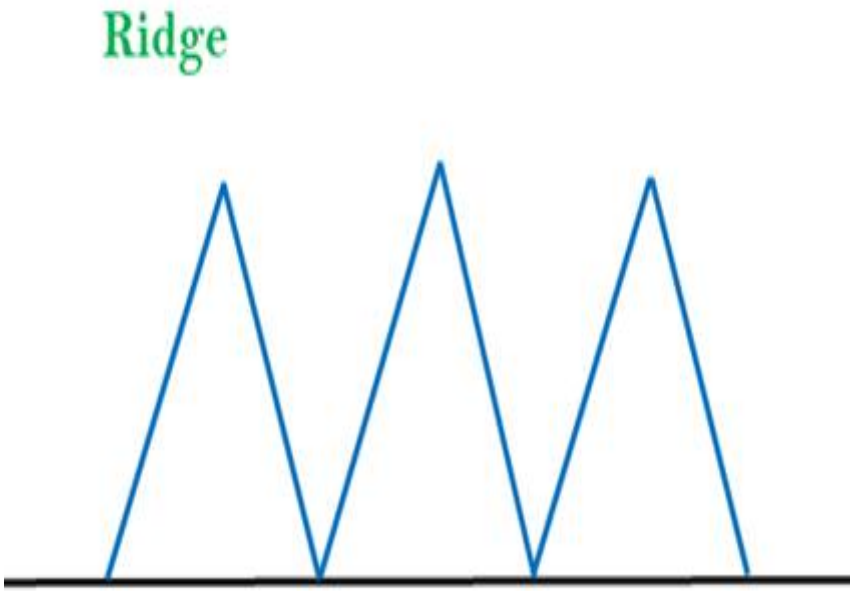


Problems in Hill Climbing Algorithm:

- **3. Ridges:** A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

- **Solution:** With the use of bidirectional search, or by moving in different directions, we can improve this problem.

Ridges



Applications of hill climbing algorithm

- The hill-climbing algorithm can be applied in the following areas:
- Marketing

  A hill-climbing algorithm can help a marketing manager to develop the best marketing plans. establish the local minima efficiently.
- Robotics
- Hill climbing is useful in the effective operation of robotics. It enhances the coordination of different systems and components in robots.
- Job Scheduling
- The hill climbing algorithm has also been applied in job scheduling.
- One of the most popular examples of a hill-climbing algorithm is the traveling salesman.

Simulate Annealing

- Instead of starting again randomly ,when the Hill climbing search gets stuck at local maxima.
- We could allow the search to take some downhill(descent) **to escape the local maxima, this is roughly the idea of Simulated annealing**
- The **inner most loop of the SA is similar to the Hill climbing.**
- Instead of picking the best move, **Simulated annealing picks up the random move.**
- If the move actually improves **the situation, it is always executed**.
- Otherwise, the algorithm **makes the move with some probability Less than 1**

- **Annealing :**comes from the **process of gradually cooling a liquid until it freezes**
- **VALUE :**corresponds to the total energy of atoms in material.
- **T :**corresponds to the temperature
- **Schedule :**determines the rate of which the temperature is lowered

Simulated Annealing algorithm

```
function SIMULATED-ANNEALING( problem, schedule )
    current ← INITIAL-STATE[ problem ]
    for t ← 1 to ∞ do
        T ← schedule[ t ]
        if T = 0 then return current
        next ← a randomly selected successor of current
        ΔE ← VALUE[ next ] – VALUE[ current ]
        if ΔE < 0 then current ← next
        else current ← next with probability e^(−ΔE/T)
```

- The probability decreases exponentially with the badness of the move, the amount delta E ,by which the evaluation is worsened.
- Second ,parameter T is also used to determine the probability
- At higher values of T “bad” bad moves are more likely to be allowed
- As T tends to zero, they become more and **more unlikely ,until the algorithm behaves more or less same to Hill climbing**
- The schedule input determines the value of T as a parameter of how many cycles already have been completed

- Simulated Annealing **provides a way to escape local maxima**,
- And is optimal and complete algorithms
- As long as given long enough cooling schedule .

Applications of SA :

- VLSI layout,
- Airline scheduling

Applications of Simulated annealing

- Factory scheduling
- VLSI layout problem solving
- Large scale optimization tasks

Simulated Annealing in Practice

– method proposed in 1983 by IBM researchers for solving VLSI layout problems (Kirkpatrick et al, *Science*, 220:671-680, 1983).

- theoretically will always find the global optimum

– Other applications: Traveling salesman, Graph partitioning, Graph coloring, Scheduling, Facility Layout, Image Processing, ...

Simulated Annealing algorithm

- A hill-climbing algorithm which never makes a move towards a lower value guaranteed to be incomplete because it can get stuck on a local maximum. And if algorithm applies a random walk, by moving a successor, then it may complete but not efficient.
- **Simulated Annealing** is an algorithm which yields both efficiency and completeness.

Simulated Annealing algorithm

- The Simulated Annealing algorithm is based upon Physical Annealing in real life.
- Physical Annealing is the process of heating up a material until it reaches an **annealing temperature** and then it will be **cooled down** slowly in order to change the material to a desired structure.
- When the material is hot, the molecular structure is weaker and is more susceptible to change. When the material cools down, the molecular structure is harder and is less susceptible to change.

Physical Annealing...

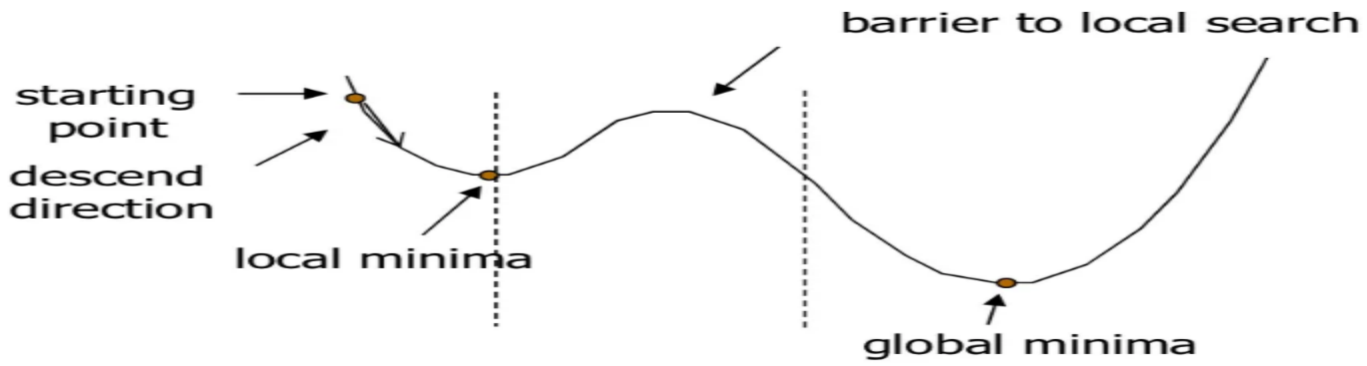
- Thermal Dynamics Equation calculates the probability that the Energy Magnitude will increase.

$$P(\Delta E) = e^{-\frac{\Delta E}{k*t}}$$

- Where ΔE - Energy Magnitude
- t - temperature
- k - Boltzmann constant.

Simulated Annealing

- It is used for optimizing parameters in a model.
- This process is very useful for situations where there are a lot of local minima.
- Gradient Descent Algorithm would be stuck at local minima and not be able to reach the global minima.

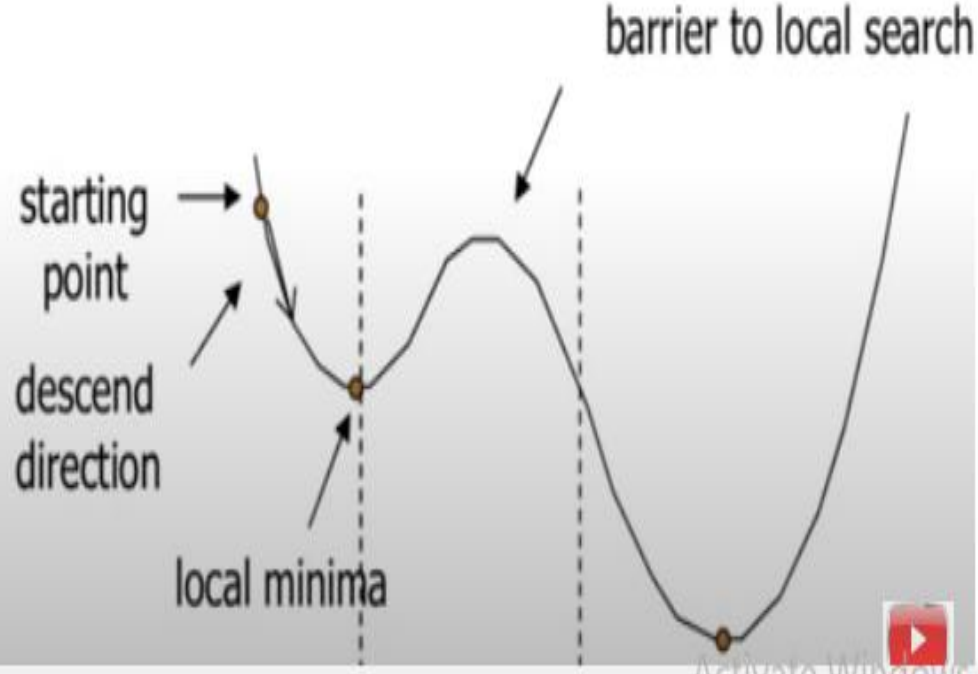


Simulated Annealing

- **Simulated Annealing** is a stochastic global search optimization algorithm and it is modified version of stochastic hill climbing.
- This algorithm appropriate for nonlinear objective functions where other local search algorithms do not operate well.
- **The simulated-annealing solution is to start by shaking hard (i.e., at a high temperature) and**
- **then gradually reduce the intensity of the shaking (i.e., lower the temperature).**
- Simulated Annealing (SA) is very useful for situations where there are a lot of local minima.

Simulated Annealing- Example- ping-pong ball

- Imagine the task of getting a ping-pong ball into the deepest crevice in a very bumpy surface.
- If we just let the ball roll, it will come to rest at a local minimum.
- The trick is to shake just **hard** enough, to bounce the ball out of local minima then the ball will reach the global minimum.



Simulated Annealing...

- Simulated annealing improves this strategy through the introduction of two tricks.
- This algorithm picks a *random* move instead of picking the best move.
- If the move improves the result then it accepts this random move, otherwise it accepts the move with some probability less than 1.

Simulated Annealing...

- Based on annealing, the algorithm uses the equation

$$e^{-\Delta D/T} > R(0, 1),$$

- Where ΔD is Change of distance,
- T is synthetic temperature (control Variable) and
- R(0,1) is a random number in the interval [0,1].

Simulated Annealing Algorithm

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

current ← *problem*.INITIAL

for $t = 1$ **to** ∞ **do**

$T \leftarrow schedule(t)$

if $T = 0$ **then return** *current*

next ← a randomly selected successor of *current*

$\Delta E \leftarrow$ VALUE(*current*) – VALUE(*next*)

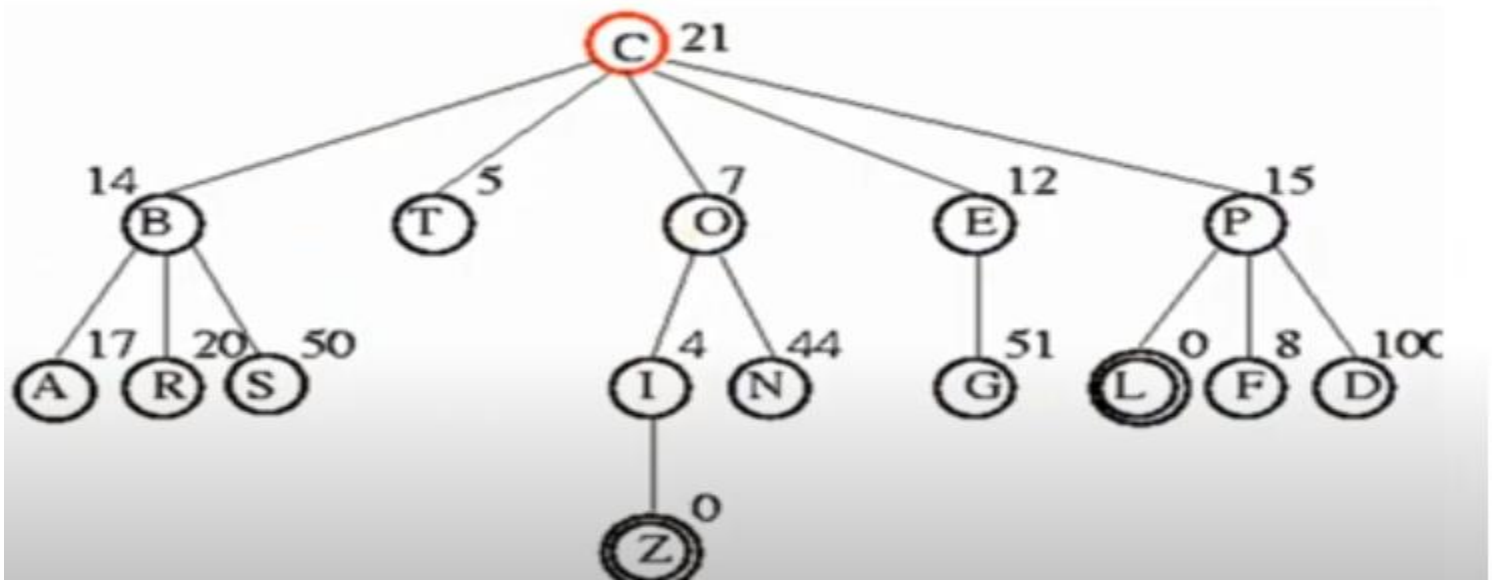
if $\Delta E > 0$ **then** *current* ← *next*

else *current* ← *next* only with probability $e^{-\Delta E/T}$

Beam Search algorithm

Beam Search Algorithm

- A heuristic search algorithm that examines a graph by extending the most promising node in a limited set is known as **beam search algorithm.**
- The number of nodes n represents the **beam width.**
- This algorithm only keeps the lowest number of nodes on open list,



LOCAL BEAM SEARCH

- Maintain **k states** rather than just one.
- Begin with **k randomly generated states**.
- Implements **parallelism concept**

Also

- In each iteration, generate all the successors of all k states.
- Stop if a goal state is found;
- otherwise Select the k best successors from the complete list and repeat..

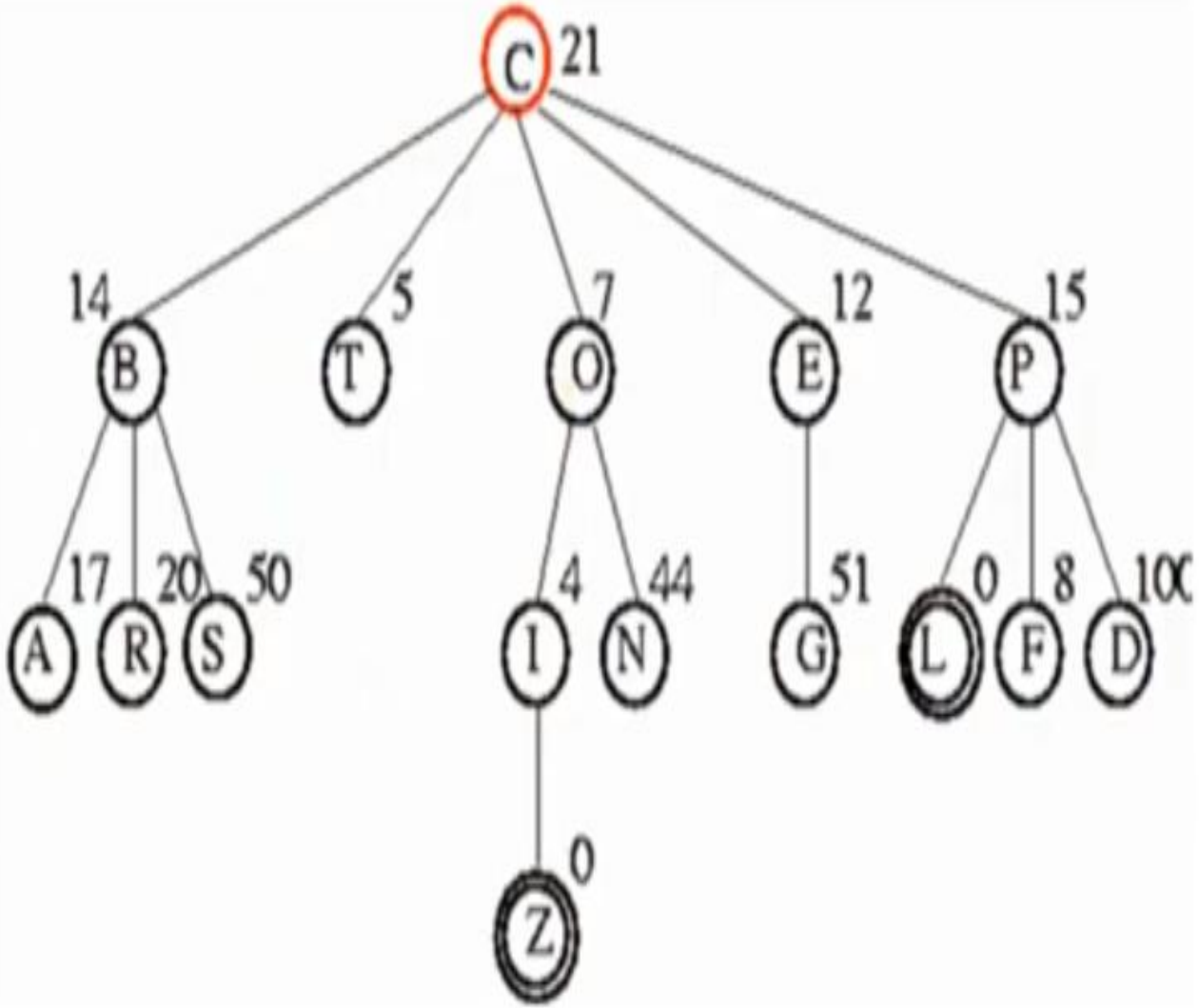
Note :The k parallel searches share the information among each other ,so as to take the search process in the most profitable direction

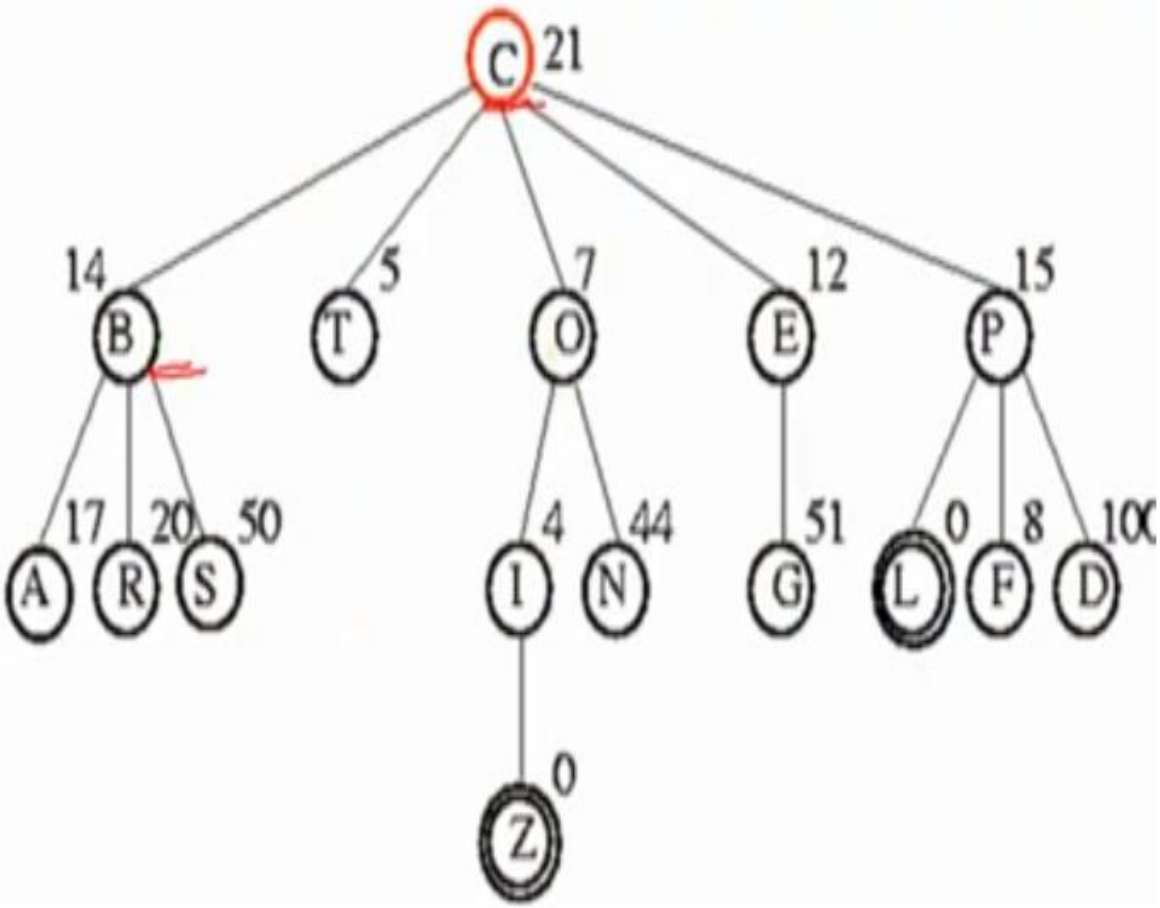
Components of Beam Search

- A beam search takes three components as its input:
- 1. The problem usually represented as graph and contains a set of nodes in which one or more of the nodes represents a goal.
- 2. The set of heuristic rules for pruning: are rules specific to the problem domain and prune unfavorable nodes from memory regarding the problem domain.
- 3. A memory with a limited available capacity
- The memory is where the ***"beam"*** is stored, memory is full, and a node is to be added to the beam, the most costly node will be deleted, such that the memory limit is not exceeded.

Example Problem

- Start State: C
- Goal State: Z and L
- n = 2 (beam size)
- Iteration 1
- Open List = {c}



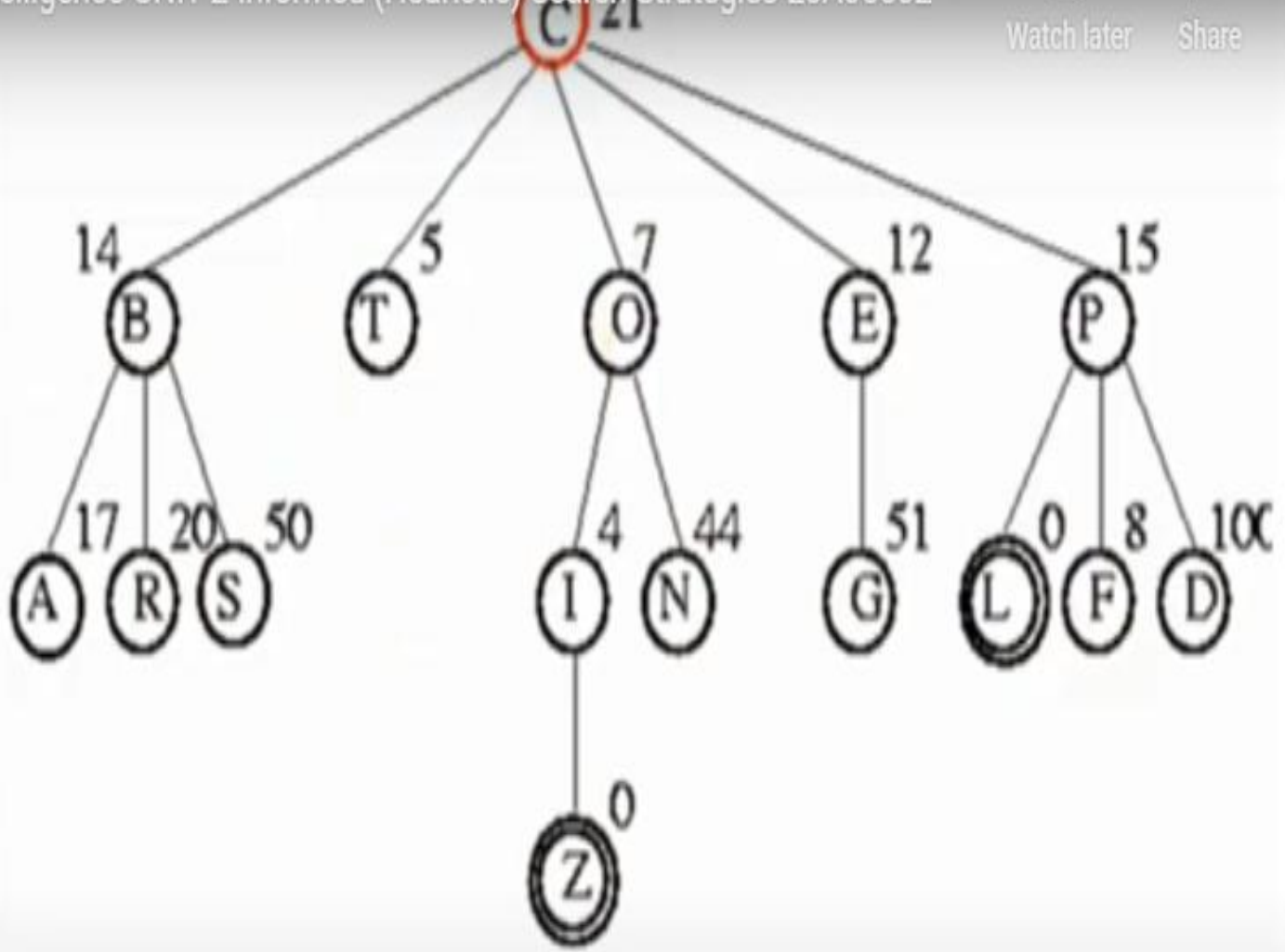


- Iteration 2
- Find successor of C
- = B T O E P
- Remove C from list, now
- Open list = {T,O}
- Iteration 3
- T has no successors, remove T from open list
- Find successor of O, replace O with I and N in open list, then
- Open list = {I, N}

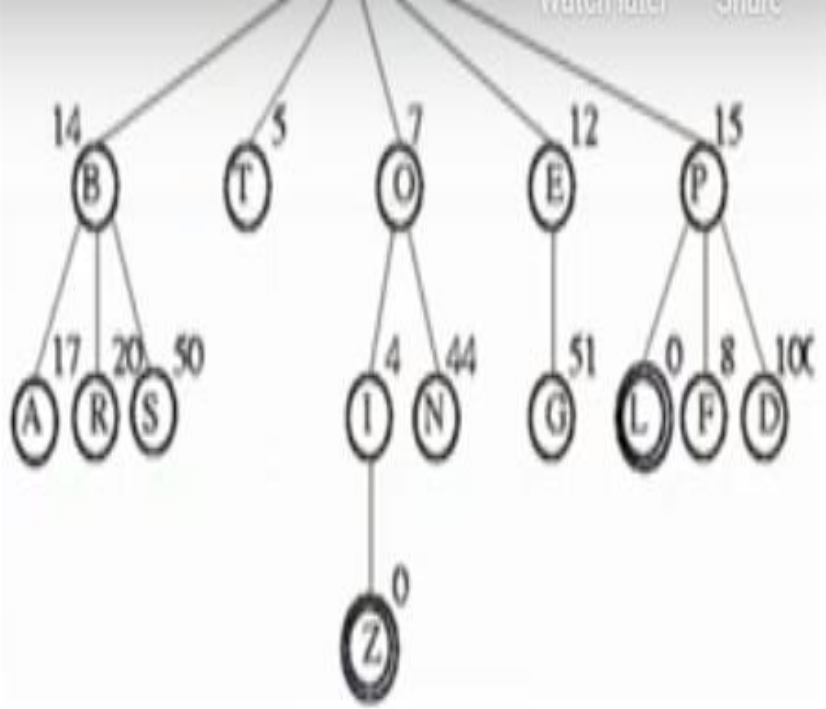
Activate Windows

Watch later Share

- Open list = {I, N}
- Iteration 4
- Find successor of I
- Z is Goal



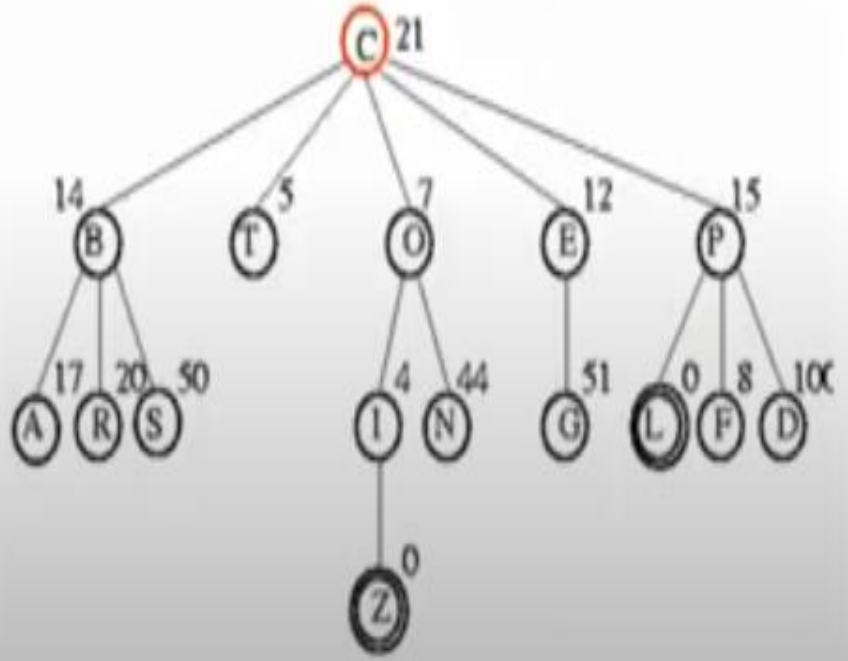
Beam Search Algorithm...



- Beam search algorithm is not complete
- It is not optimal
- The time complexity: The worst-case time = $O(B*m)$
- The space complexity: The worst-case space complexity = $O(B*m)$
- B is the beam width, and m is the maximum depth of any path in the search tree.

Beam Search Algorithm...

- If the beam size is 1 then this algorithm is Best First Searching
- When compared to A* searching, the time and space complexity will be very much reduced.



Local beam search algorithm

Local Beam Search

Algorithm:

```
function BEAM_SEARCH(problem, k) returns a solution state
    start with k randomly generated states
    loop
        generate all successors of all k states
        if any of them is a solution then return it
        else select the k best successors
```

Steps

- **Initialization:** Start with **multiple initial solutions.**
- **Evaluation:** Evaluate the quality of each initial solution.
- **Neighbor Generation:** Generate neighboring solutions for all the current solutions.
- **Selection:** Choose the top solutions based on the improvement in the objective function.
- **Termination:** Continue iterating until a termination condition is met.
- Local beam search effectively avoids local optima because it maintains diversity in the solutions it explores. However, it requires more memory to store multiple solutions in memory simultaneously.

Stochastic beam search

- Stochastic beam search is **an alternative to beam search, which, instead of choosing the best k individuals, selects k of the individuals at random; the individuals with a better evaluation are more likely to be chosen**.
- This is done by making the probability of being chosen a function of the evaluation function.

Genetic algorithms

- Is a variation is Stochastic local beam search: in which successor states are generated by combining 2 parent states rather than modifying a single state.
- Selects successors at random instead of selecting the best successors.
- A close analogy to the natural selection.**(arrived from Darwins philosophy/theory)**

- **A slight variation of stochastic local beam search is the Genetic algorithms**
- **Based on Evolution**
- Genetic algorithms are based on combining 2 parent states

Genetic Algorithm

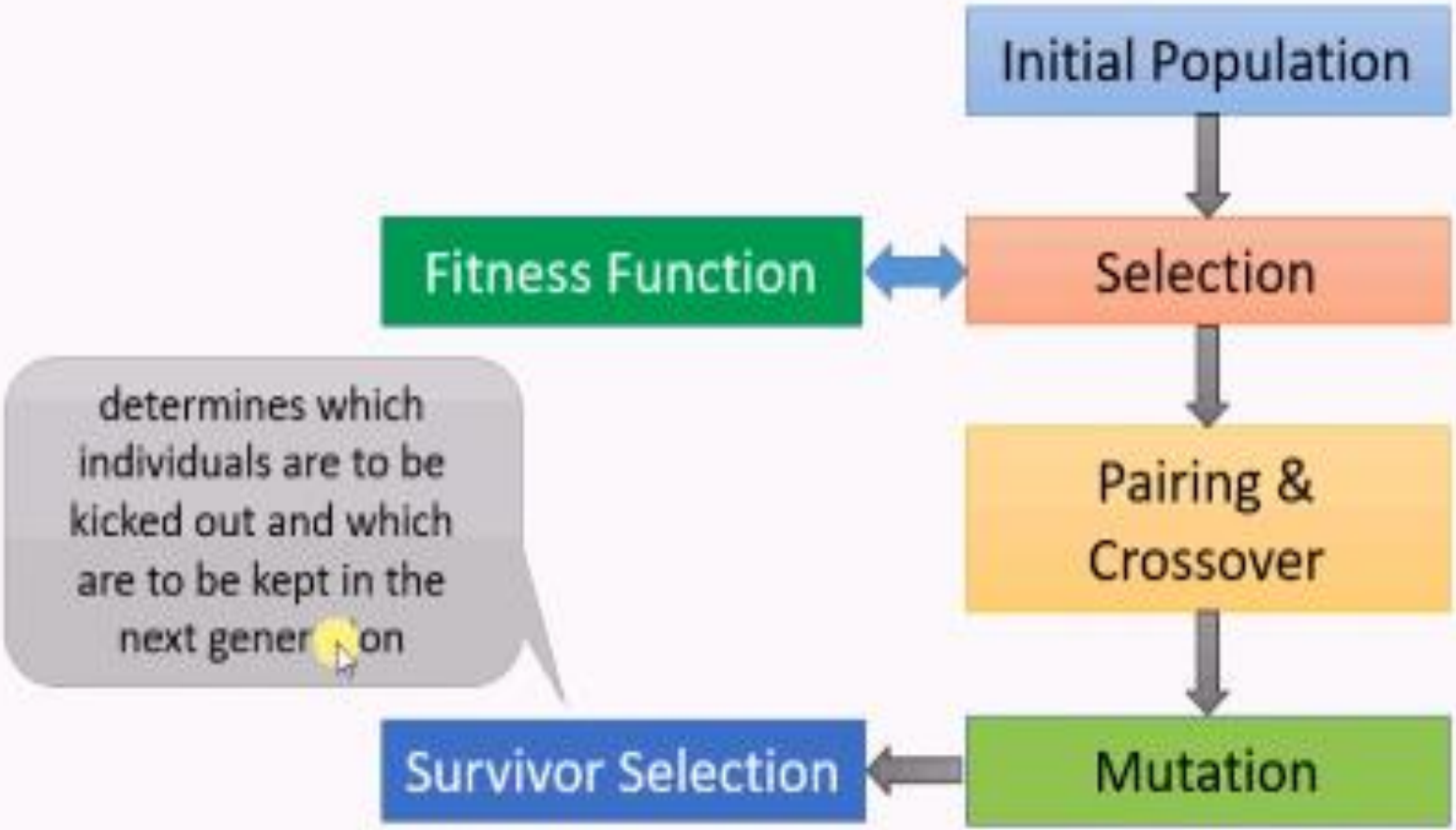
- Population
- Fitness
- Selection
- Crossover
- Mutation

- Optimization Algorithm
- Based on natural phenomenon
- Nature inspired approach based on Darwin's law of Survival of the fittest and bio-inspired operators such as Pairing Crossover and Mutation.
- frequently used to find optimal or near-optimal solutions to difficult problems

- A genetic algorithm (GA) is a powerful search and optimization technique inspired by the process of natural selection and evolution.

- Instead of trying every possible solution, it intelligently "evolves" a population of potential solutions over many generations to find the best one.

Concept


Initial Population
Fitness Function
Selection
determines which individuals are to be kicked out and which are to be kept in the next generation
Pairing & Crossover
Survivor Selection
Mutation

How it works:

- **Initial Population:** It starts with a group of random, potential solutions, like a collection of individuals. Each solution is represented as a "chromosome."

- **Fitness Function:** Each solution is evaluated using a fitness function, which determines how good or "fit" it is. The better the solution, the higher its fitness score.

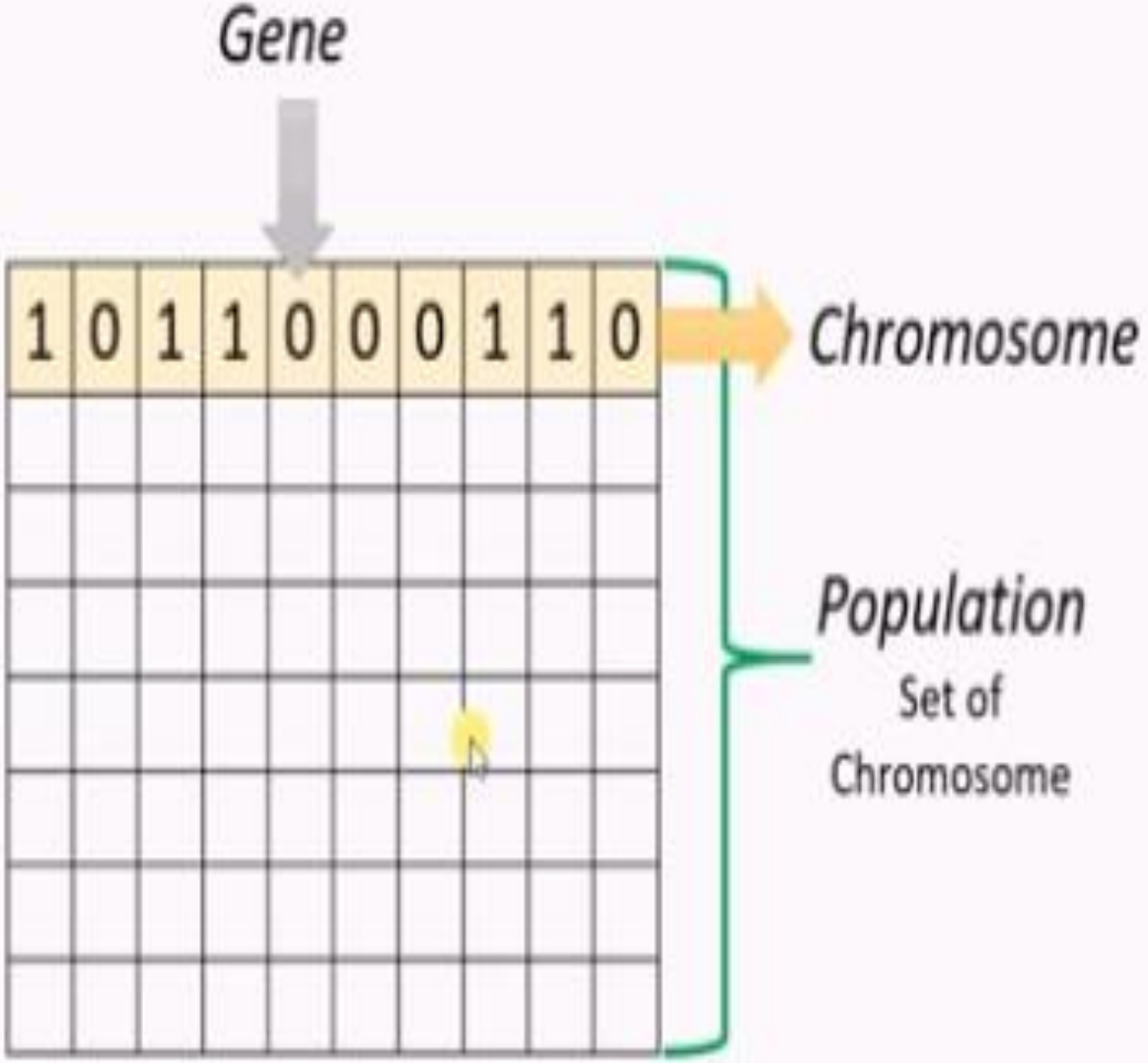
- **Selection:** The algorithm selects the fittest individuals from the current population to be the "parents" of the next generation. The best solutions have a higher chance of being chosen.

- **Crossover:** The parents' chromosomes are combined in a process called crossover to create new offspring. This allows the algorithm to mix the best traits from different solutions.

- **Mutation:** A small, random change is made to the offspring's chromosome. This introduces new variations and helps the algorithm avoid getting stuck in a single, non-optimal solution

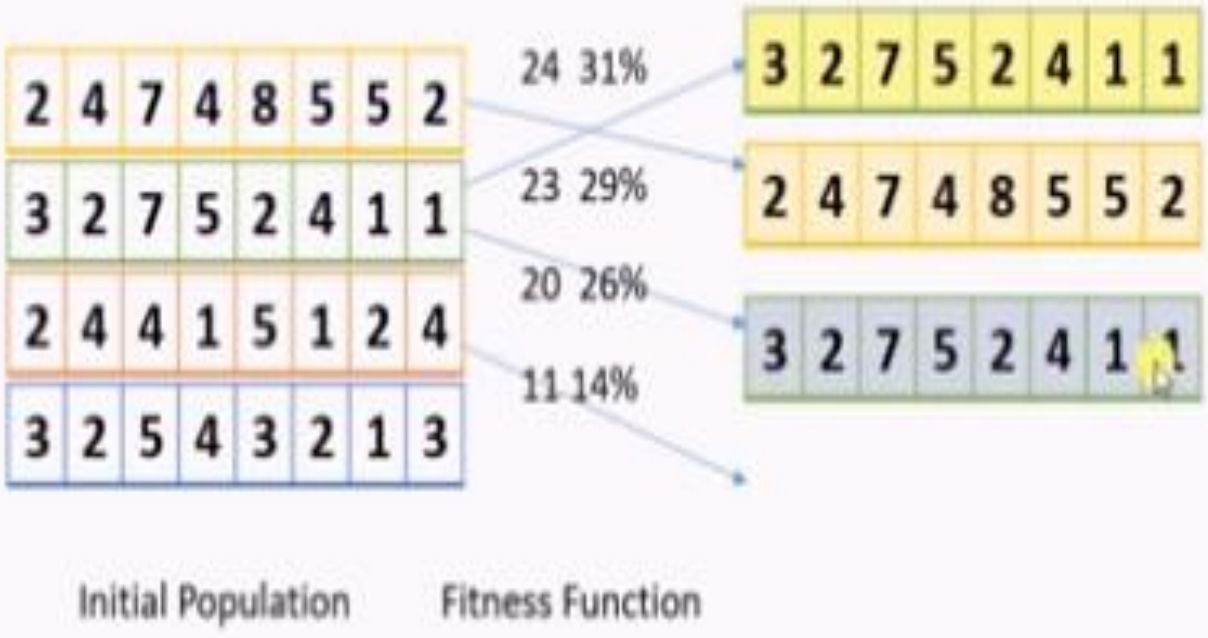
Terminology

- Population
- Chromosomes
- Gene



Fitness Function

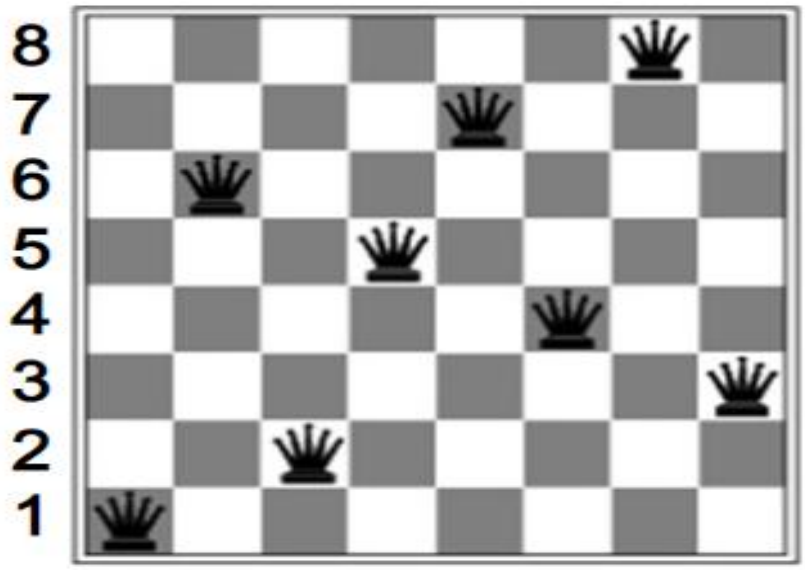
The *fitness function* is the function you want to optimize. Function which takes the solution as input and produces the suitability of the solution as the output.



Fitness Function

- For the 8-queens problem we use the number of *nonattacking* pairs of queens, which has a value of 28 for a solution.
- The values of the four states are 24, 23, 20, and 11
- Fitness function : number of non-attacking pairs of queens
- (min = 0, max = 8 × 7/2 = 28)
- 24/(24+23+20+11) = 31%
- 23/(24+23+20+11) = 29% etc.

String representation



String representation
16257483

Genetic algorithms



4 states for 8-queens problem

2 pairs of 2 states randomly selected based on fitness. Random crossover points selected

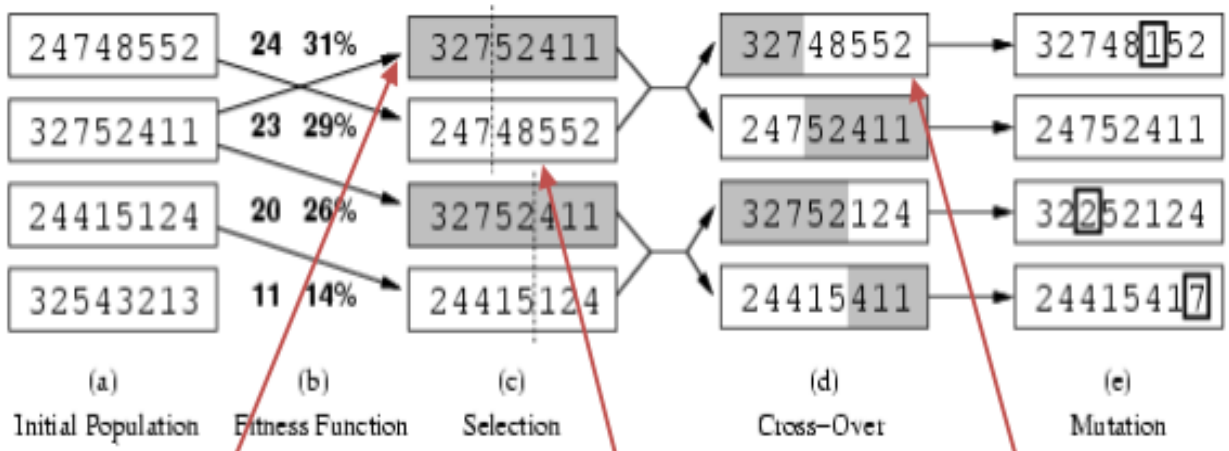
New states after crossover

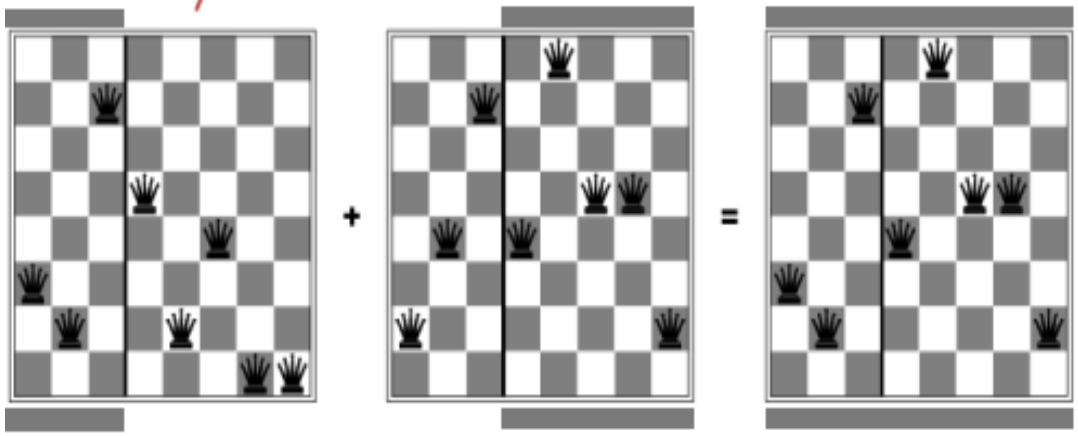
Random mutation applied

- Fitness function: number of non-attacking pairs of queens (min = 0, max = $8 \times 7/2 = 28$)
- 24/(24+23+20+11) = 31%
- 23/(24+23+20+11) = 29% etc

Genetic Algorithm for 8 Queens

Fitness function: # non-attacking pairs
(min = 0, max = 8×7 / 2 = 28)



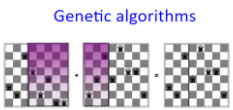


Population fitness = 24+23+20+11 = 78

P(Gene-1 is chosen)
= Fitness of Gene-1 / Population fitness
= 24 / 78 = 31%

P(Gene-2 is chosen)
= Fitness of Gene-2 / Population fitness
= 23 / 78 = 29%

Genetic algorithm



function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual
  inputs: *population*, a set of individuals
    FITNESS-FN, a function that measures the fitness of an individual

  repeat
    new_population ← empty set
    for $i = 1$ **to** SIZE(*population*) **do**
      x ← RANDOM-SELECTION(*population*, FITNESS-FN)
      y ← RANDOM-SELECTION(*population*, FITNESS-FN)
      child ← REPRODUCE(x, y)
      if (small random probability) **then** *child* ← MUTATE(*child*)
      add *child* to *new_population*
    population ← *new_population*
  until some individual is fit enough, or enough time has elapsed
  return the best individual in *population*, according to FITNESS-FN

---

function REPRODUCE(x, y) **returns** an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, $c + 1$, n))

Comments on Genetic Algorithms

- Genetic algorithm is a variant of "stochastic beam search"
- Positive points
  - Random exploration can find solutions that local search can't
    - (via crossover primarily)
  - Appealing connection to human evolution
    - "neural" networks, and "genetic" algorithms are **metaphors**!
- Negative points
  - Large number of "tunable" parameters
    - Difficult to replicate performance from one problem to another
  - Lack of good empirical studies comparing to simpler methods
  - Useful on some (small?) set of problems but no convincing evidence that GAs are better than hill-climbing w/random restarts in general

Search with partial information

- If the environment is not fully observable or deterministic, then the following types of problems occur:

1. Sensorless problems

2. Contingency problems

3. Exploration problems

- .

1.Sensorless problems

- If the agent has no sensors, then the agent cannot know it's current state, and hence would have to make many repeated action paths to ensure that the goal state is reached regardless of it's initial state

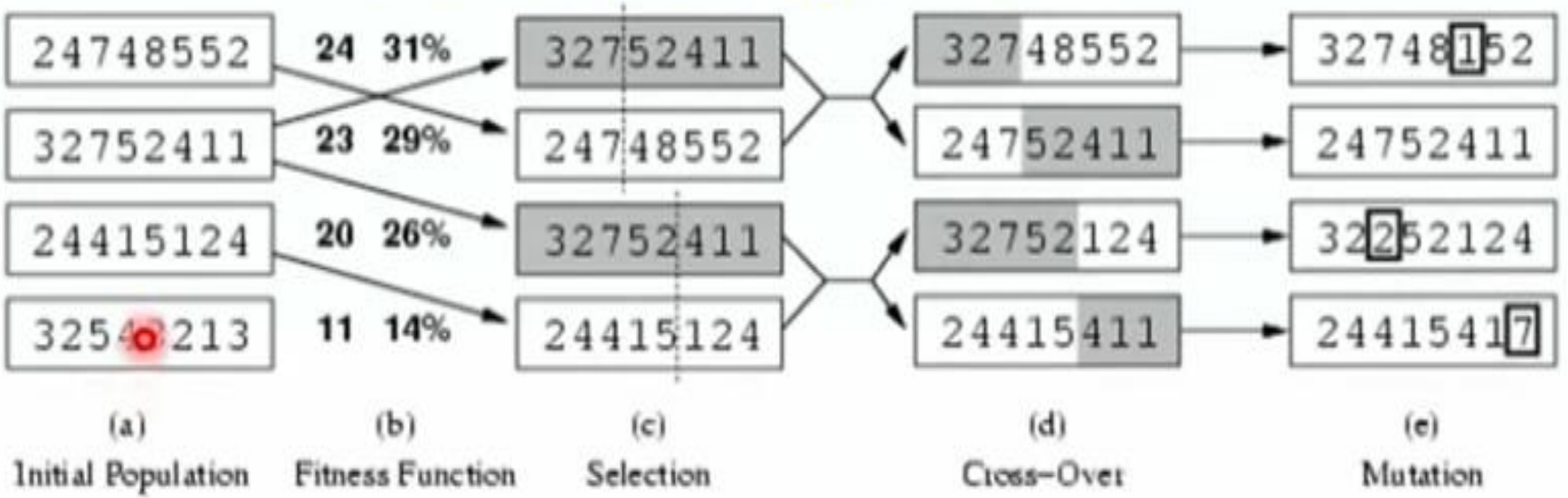
2. Contingency problems

- This is when the environment is **partially observable or when actions are uncertain.**
- Then after each action the agent needs to verify what effects that action has caused.
- Rather than planning for every possible contingency after an action, it is usually better to start acting and see which contingencies do arise.
- This is called **interleaving of search and execution**.

Exploration problems

- **3.** This can be considered an extreme case of contingency problems:
- when the states and actions of the environment is unknown, the agent must act to discover them.

Genetic algorithms



4 states for 8-queens problem

2 pairs of 2 states randomly selected based on fitness. Random crossover points selected

New states after crossover

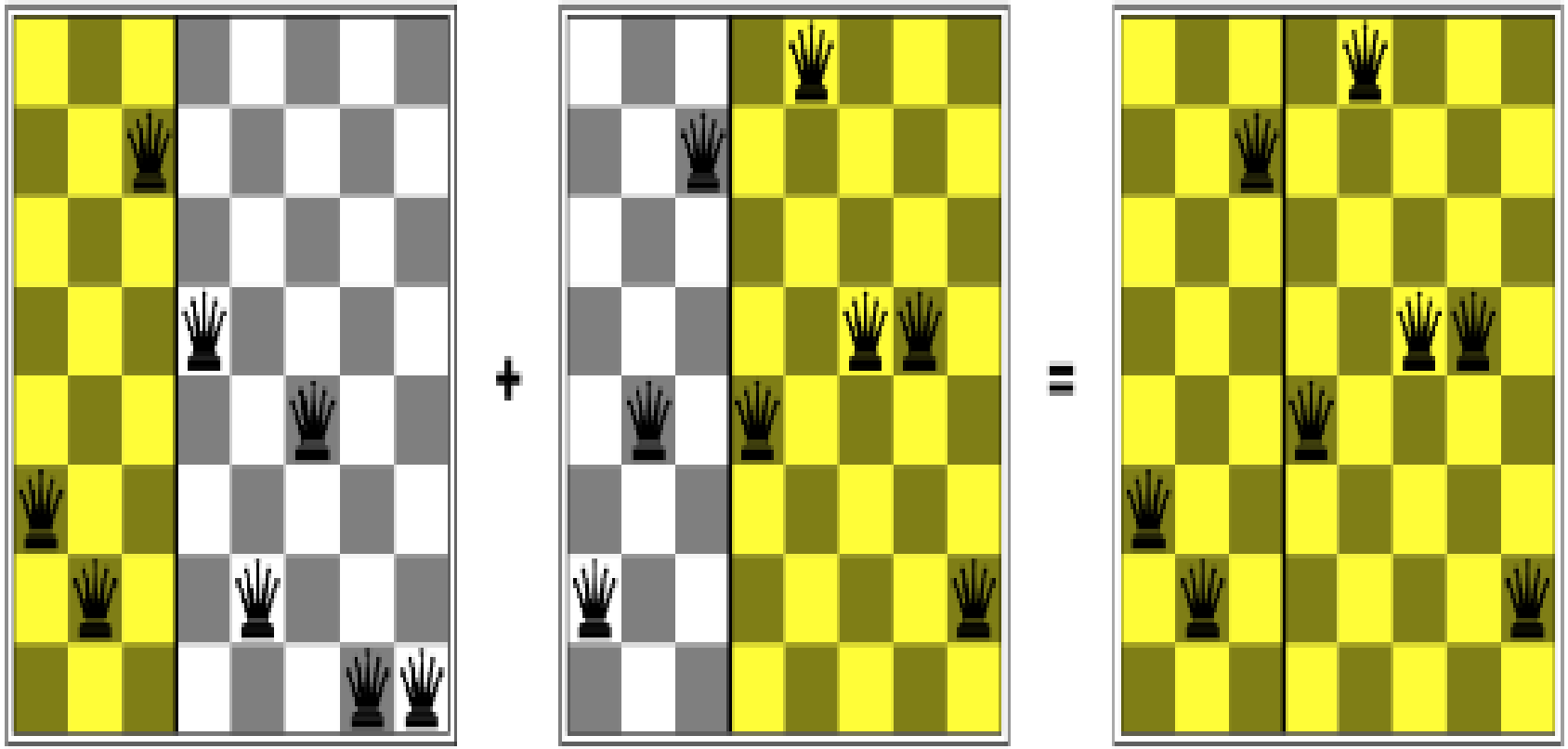
Random mutation applied

- Fitness function: number of non-attacking pairs of queens (min = 0, max = 8 × 7/2 = 28)
- 24/(24+23+20+11) = 31%
- 23/(24+23+20+11) = 29% etc

Genetic algorithms

Crossover

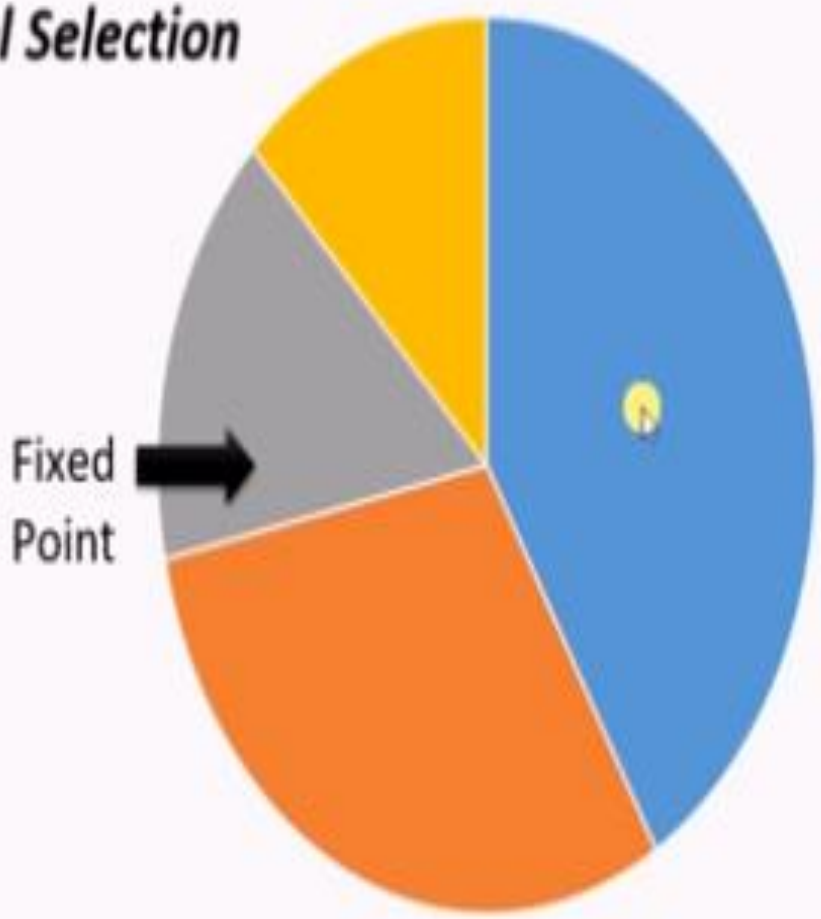
Taken from the edX course ColumbiaX: CSMM.101x Artificial Intelligence (AI)



Generate successors from pairs of states.

Parent Selection

- ***Roulette Wheel Selection***



Chromosome	Fitness Value
A	8.9
B	6.5
C	3.4
D	2.7

Parent Selection

- ***Rank Selection*** : Remove the concept of a fitness value while selecting a parent. Every individual in the population is ranked according to their fitness.

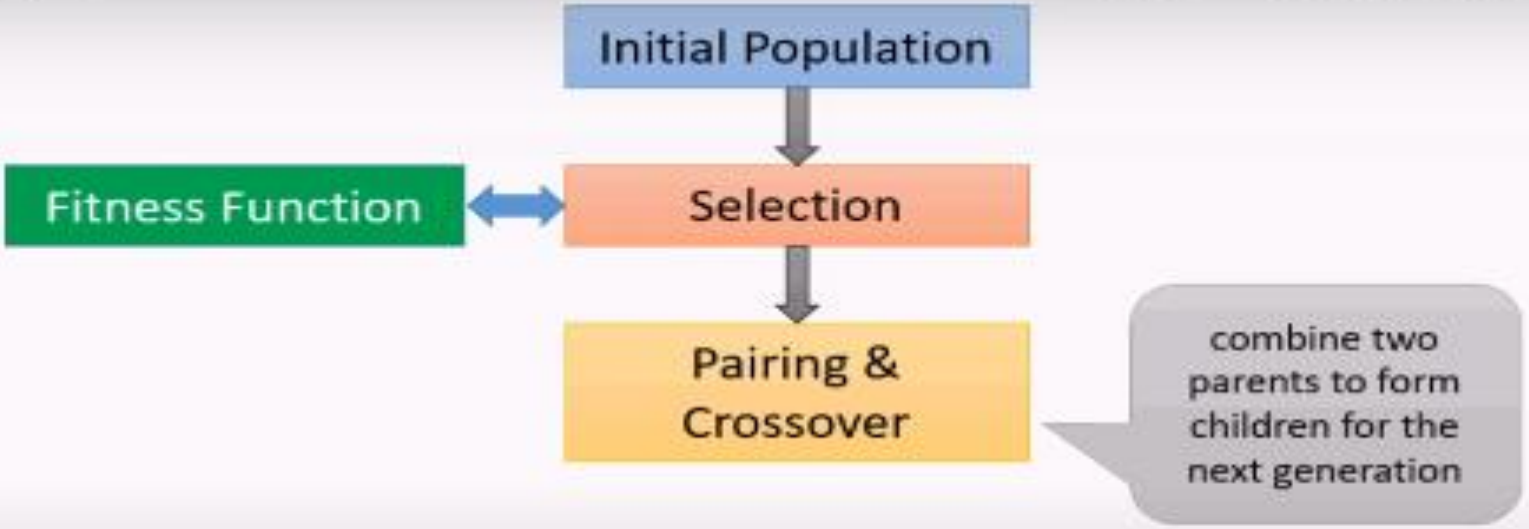
Chromosome	Fitness Value	Rank
A	9.8	1
B	7.9	2
C	2.4	5
D	5.2	3
E	4.1	4

Activate Windows

Parent Selection

- ***Stochastic Universal Sampling (SUS)*** : Multiple fixed points, all the parents are chosen in just one spin of the wheel.
- ***Tournament Selection*** : Select K individuals from the population at random and select the best out of these to become a parent, same process is repeated for selecting the next parent.

Concept



Pairing and Crossover

- *One Point Crossover* : A random crossover point is selected and the tails of its two parents are swapped to get new off-springs.



Mutation

- *Bit Flip Mutation* : Select one or more random bits and flip them.



- *Swap Mutation*



Survivor Selection

- *Age Based Selection*
- *Fitness Based Selection*

Chromosome	Fitness Value
A	7.8
B	7.9
C	2.4
D	5.2
E	4.1

Existing Population

Chromosome	Fitness Value
P	8.8
Q	8.6

Offspring's Population

Chromosome	Fitness Value
P	8.8
B	7.9
A	7.8
C	2.4
D	5.2
E	4.1
Q	8.6

New Population

Genetic algorithms

function GENETIC-ALGORITHM(population, fitness-function)
returns an individual

repeat

initialize new-population **with** $\emptyset$

for i=1 to size(population) **do**

x = random-select(population,fitness-function)
x = random-select(population,fitness-function)
child = cross-over(x,y)
mutate (child) with a small random probability
add child to new-population

population = new-population

until some individual is fit enough or enough time has elapsed

return the best individual in population w.r.t. fitness-function

Advantages of Genetic Algorithm

- Does not require any derivative information.
- faster and more efficient as compared to the traditional methods.
- Optimizes both continuous and discrete functions and also multi-objective problems.
- Provides a list of "good" solutions and not just a single solution.
- Useful when the search space is very large and there are a large number of parameters involved.

Limitations of Genetic Algorithm

- Computationally expensive as Fitness value is calculated repeatedly.
- Not suited for all problems, especially problems which are simple and for which derivative information is available.
- GA may not converge to the optimal solution, if not implemented properly.

APPLICATIONS:

- Engineering, Economics, Manufacturing, Artificial Intelligence, Operations research, Space science, Agriculture, Physics, Chemistry, Bioinformatics, Medical science,
- Diagnosis of genetic diseases
- Personalization of medical treatments
- Gene therapies
- Vaccine development
- Risk predictor.

Example: The Traveling Salesperson Problem (TSP)

- TSP is a classic example of an optimization problem where GAs excel. The goal is to find the shortest possible route that visits a set of cities exactly once and returns to the origin city.

GA Approach to TSP:

- **Representation:** Each individual (chromosome) in the population is a list of city numbers representing a possible tour, for example, [City A, City C, City B, City D].

- **Population Initialization:** A random set of possible tours is created.

- **Fitness Function:** The fitness of each tour is calculated as the inverse of its total distance. A shorter tour has a higher fitness score.

- **Selection:** Tours with shorter total distances (higher fitness) are chosen as parents.

- **Crossover:** The parent tours' city sequences are combined to create new offspring tours. For instance, a part of one parent's sequence might be swapped with a part of another's to create a new, potentially shorter, route.

- **Mutation:** A small, random change is made to an offspring's tour, such as swapping the positions of two cities. This could help the algorithm find a better local arrangement of cities.